

- Problemas frecuentes en el desarrollo de software
 - 1. La versión final del producto no satisface las necesidades del cliente
 - 2. No es fácil extenderlo y/o adaptarlo. Agregar más funcionalidad en otra versión es casi una misión imposible
 - 3. Mala documentación
 - 4. Mala calidad
 - 5. Más tiempos y costos que los presupuestados
- Cuando nos va bien...
- Ingeniería de Software
- Disciplinas de la Ingeniería de Software
 - Disciplinas Técnicas
 - Disciplinas Gestión
 - Disciplinas de Soporte
 - El proceso de Software
 - Personas con habilidades, entrenamiento y motivación
 - Procedimientos y métodos
 - Herramientas y equipos
 - Proceso
- Procesos definidos vs procesos empíricos
 - Proceso definido
 - Proceso empírico
- Ciclos de vida
 - Tipos de ciclo de vida
 - ¿Qué relación hay entre ciclos de vida y procesos?
 - Resumen para examen
 - ¿Qué es un ciclo de vida?
 - ¿Para qué sirve?
 - ¿Qué tipos existen?
 - Diferencia entre ciclo de vida de producto y proyecto:
- SCM Gestión de la Configuración de Software
 - ¿Qué elementos se gestionan en el SCM?
 - Componentes típicos del SCM
- Software en contexto
 - **PROCESO**
 - **PROYECTO**
 - **PERSONAS**
 - **HERRAMIENTAS**
 - **PRODUCTO**

- Cuando pensamos en Software... en qué pensamos?
 - Programas
 - Procedimientos
 - Reglas
 - Documentación
 - Datos
- Cambios en el software
 - Qué rol cumple SCM frente a los cambios
 - Funciones clave:
 - ¿Por qué se dice que SCM es una **disciplina paraguas**?
 - ¿Cuándo actúa SCM?
 - SCM como **disciplina de soporte y protección**
 - 1. **Administración de Configuración de Software (SCM)**
 - 2. **Control de Calidad del Proceso**
 - 3. **Control de Calidad del Producto**
 - 4. **Pruebas de Software**
 - Definición de SCM por ANSI/ IEEE 828, 1990
 - ¿Por qué es necesario gestionar la configuración del software?
 - Conceptos clave de Gestión de la Configuración de Software
 - Ítem de configuración
 - Repositorio
 - Línea Base
 - Configuración del software
 - Versión vs Variante
 - Ramas
 - 4 Actividades Fundamentales de la Gestión de Configuración
 - 1. Identificación de Ítems de Configuración
 - 2. Control de cambios
 - 3. Informes de estado
 - 4. Revisiones y Auditorías
 - Plan de Gestión de Configuración
 - 1. **Introducción**
 - 2. **Organización del proyecto**
 - 3. **Fases del proyecto y Hitos**
 - 4. **Identificación de Configuración**
 - 5. **Gestión de Interfaces**
 - 6. **Control de Cambios (Change Control)**
 - 7. **Contabilidad de Estado (Status Accounting)**
 - 8. **Auditorías de Configuración**

- 9. **Gestión de Subcontratistas**
- Reglas(Algunas)
- Gestión de la Configuración en Agile
 - Qué pasa con el comité de cambios en Agil
 - Qué ítem hay que mantener
 - ¿Qué pasa con las auditorías?
- Definición bien completa SCM
 - **Conceptos clave**
 - **Actividades Fundamentales**
 - **Para qué sirve**
 - **Fases** de implementación de SCM
 - **SCM en contextos ágiles**
- Reglas y Colaciones de CM
 - Regla 1
 - Regla 2
 - Regla 3
 - Regla 4
 - Regla 5
 - Regla 6
 - Regla 7
 - Regla 8
 - Regla 9
 - Regla 10
 - Corolario 1
 - Corolario 2
 - Corolario 3
- Filosofía Ágil. Manifiesto Ágil
 - ¿Qué es un proceso empírico?
 - Manifiesto Ágil
 - 12 Principios del Manifiesto Ágil
 - Principios que reflejan el pensamiento empírico
 - Ágil es una filosofía, no una metodología
 - Valores de los equipos ágiles
 - Requerimientos Ágiles
 - User Stories vs Requerimientos ágiles
 - ¿Cómo se expresan los requerimientos? → User stories
 - 3C
 - ¿Por qué usar historias?
 - Cómo escribir buenas historias (regla INVEST + formato)

- El costo del enfoque tradicional BRUF (Big Requirements Up Front)
- Gestión Ágil de requerimientos de software
 - Backlog y prioridad por valor
 - Historias de usuario como técnica central de gestión
- Conversación, una pieza clave
- Tipos de requerimientos
- Triangulo Negocio-Usuario-Software
- Triangulo de hierro, **Tradicional (plan-driven) vs Ágil (value-driven)**
- Imagen de resumen
- Principios ágiles relacionados a los requerimientos ágiles
- User Stories: qué son, 3C y por qué importan
 - **Las 3C: Card, Conversation, Confirmation (lo más importante es la conversación)**
- El product Owner prioriza el product backlog
- “User stories: porciones verticales” (vertical slices)
- Modelado de roles, Personas y “Personajes extremos”
- “Usuarios representantes / Proxies”: por qué **no son ideales**
- Criterios de aceptación
- Pruebas de aceptación
- Relación y diferencias
- Algo más sobre las User Stories...
- ¿Story, Épica y Theme: en qué se diferencian?
- Spikes: qué son, para qué sirven y cuándo usarlos
- Estimación Ágil y Tradicional
 - Tips clave sobre estimación
 - Estimación ágil vs tradicional
 - Estimación relativa
 - ¿Qué son los Story Points y por qué son “relativos”?
 - ¿Cómo estimar en ágil?
 - Tamaño en Agile
 - Tamaño ≠ esfuerzo (ni tiempo)
 - Cómo armar el tamaño
 - Dónde entra “lo tradicional”
 - Velocity
- Poker Panning
- Gestión de Productos
 - Diccionario rápido
 - **MVP – Minimum Viable Product (Producto Mínimo Viable)**
 - **MMF – Minimum Marketable Feature (Característica Mínima

Comercializable)**

- **MMP – Minimum Marketable Product (Producto Mínimo Comercializable)**
- **MMR – Minimum Marketable Release (Release Mínimo Comercializable)**
- Errores comunes
- Valor vs Desperdicio
- Lean Startup: Build–Measure–Learn
- Construir un MVP
- La audiencia de cero
- Supuesto de "salto de fe"
- Preparar un MVP
- Características de MVP
- Matriz de priorización para el MVP
- Componentes de proyecto de desarrollo de Software
 - Proceso de software
 - Proceso Definido
 - Proceso Empírico
 - Patrón de conocimiento en procesos empíricos
 - Ciclos de vida: producto vs proyecto
 - Tipos básicos de ciclo de vida del proyecto
 - Qué es un proyecto
 - Características de los proyectos
 - Qué es la administración de proyectos
 - La **triple restricción** (alcance–tiempo–costo) y la calidad
 - Procesos y ciclos de vida (cómo se relacionan)
 - Desarrollo incremental y _release planning_
 - Rol del líder de proyecto
 - Plan de proyecto (no confundir con roadmap)
 - Qué es un plan de proyecto
 - Definición del objetivo del proyecto
 - Definición del alcance
 - Proceso y ciclo de vida
 - Estimación de software
 - Tradicional vs Ágil
 - Gestión de riesgos
 - Asignación de recursos y organización
 - Programación (cronograma)
 - Controles del proyecto
 - Métricas
 - Monitoreo y Control

- “¿Cómo se atrasa un proyecto?” — “De a un día por vez”
- Tres factores TOP de éxito
- Causas frecuentes de fracaso
- Scrum (ya estamos por finalizar al fin, interminable esto lpm)
 - Resumen de la imagen
 - Roles/ Responsabilidades
 - Artefactos y sus compromisos
 - Timeboxes de Scrum
 - Pilares y valores que enmarcan todo
 - DoR vs DoD (la duda frecuente)
 - Capacidad del equipo en un Sprint
 - Ambiente de trabajo
 - Herramientas
 - Sprint Burndown y gráficos de seguimiento
 - Múltiples niveles de planificación
 - ¿Qué y cuándo estimar?
 - Planificación de Portafolio
 - Planificación de Producto
 - Planificación de Release
 - Niveles de granularidad
 - Releases y cadencia de Sprints
 - Sprint Planning (planificación de iteración)
 - Daily Scrum (planificación diaria)
 - RTF – Running Tested Features
 - Capacidad del equipo (Capacity)
 - Velocity (Velocidad)

Problemas frecuentes en el desarrollo de software

1. La versión final del producto no satisface las necesidades del cliente

- Requerimientos mal definidos o incompletos
- Cambios en el negocio durante el desarrollo
- Poca participación del cliente en el proceso
- Comunicación deficiente entre técnicos y stakeholders

Este problema se combate con buenas prácticas como

- Historias de usuario bien escritas
- Involucramiento activo del cliente

- Validaciones tempranas

2. No es fácil extenderlo y/o adaptarlo. Agregar más funcionalidad en otra versión es casi una misión imposible

Esto se refiere a un problema de **diseño pobre** o *arquitectura rígida*.

Un software **mal diseñado** puede cumplir con su función original, pero **no puede evolucionar fácilmente**. Cada nuevo cambio implica modificar muchas partes del sistema, lo que:

- Genera errores nuevos.
- Aumenta los tiempos de desarrollo.
- Aumenta los costos.

Esto ocurre porque:

- Uso de código "hardcodeado" o muy acoplado.
- Falta de principios de diseño como SOLID.
- No aplicar arquitectura en capas, modularidad ni reutilización.

✓ Éxito

Un buen software no solo funciona: debe **ser mantenible, adaptable y escalable**.

3. Mala documentación

Scrum NO dice que no se documenta

⚠ Peligro

“Software funcionando sobre documentación exhaustiva”

Esto significa: priorizar valor real, pero **no eliminar la documentación**, solo hacerla **útil y necesaria**, sin burocracia.

Mala documentación:

- Documentación desactualizada.
- Incompleta.
- Ambigua o difícil de entender.
- Que nadie lee ni usa.

Cuando hay que mantener el software o incorporar nuevos desarrolladores, **no hay guía confiable**. Esto genera errores, demoras, frustración y pérdida de conocimiento.

La documentación debe responder a las preguntas: ¿Qué hace el software?, ¿Cómo lo hace?, ¿por qué fue diseñado así?, ¿Quién lo usa?, ¿Cómo se prueba?

4. Mala calidad

El software tiene defectos que afectan su comportamiento, usabilidad, seguridad o rendimiento.

- Código mal escrito.
- Pruebas insuficientes o inexistentes.
- Mal diseño de interfaz.
- No cumplir los criterios de aceptación.

Calidad ≠ solo que “no tenga bugs”

También implica que:

- Sea fácil de usar.
- Cumpla con lo prometido.
- Sea confiable y eficiente.

✓ Éxito

La calidad **no se prueba al final**, se **construye desde el inicio** con buenas prácticas de desarrollo y pruebas continuas.

5. Más tiempos y costos que los presupuestados

Este es uno de los **problemas más históricos** en el desarrollo de software. Frederick Brooks lo describe muy bien en *The Mythical Man-Month*.

Por qué ocurre:

- Estimaciones mal hechas (por intuición en lugar de métricas).
- Requerimientos que cambian (scope creep).
- Subestimación de tareas complejas.
- Falta de planificación.
- Mala gestión de equipo y tiempos.

El proyecto **genera pérdidas** o **frustraciones** por no cumplir las expectativas.

Buenas prácticas para evitar esto:

- Estimaciones colaborativas (ej. Planning Poker).
- Reuniones de refinamiento.

- MVP claro.
- Feedback constante.
- Gestión ágil del backlog.

Problema	Causa principal	Consecuencia	Solución recomendada
El producto no satisface al cliente	Requerimientos mal definidos, mala comunicación	Rechazo del sistema, rehacer funcionalidades	User Stories bien escritas, feedback temprano
No es extensible ni adaptable	Diseño rígido, acoplamiento, sin arquitectura	Costoso de mantener, riesgo al modificar	Aplicar principios de diseño, modularidad
Mala documentación	Falta de actualización, poco enfoque en documentación útil	Dificulta mantenimiento y colaboración	Documentación clara, mínima y viva
Mala calidad	Pruebas insuficientes, mal diseño, apuro	Bugs, fallos, mala experiencia de usuario	Integrar QA desde el inicio, testing continuo
Exceso de tiempo/costo	Estimaciones erróneas, cambios constantes	Pérdida de confianza, pérdidas económicas	Estimaciones ágiles, MVP, gestión de alcance

Cuando nos va bien...

1. Involucramiento del usuario (15.9%)

Cuando el usuario (o el cliente) está presente y participa activamente (por ejemplo, en las reuniones de planificación, en las revisiones, dando feedback), el resultado final tiende a alinearse mejor con lo que realmente necesita.

2. Apoyo de la gerencia (13%)

El apoyo de la gerencia es crucial. Si la dirección de la empresa respalda el proyecto con los recursos necesarios (tiempo, presupuesto, herramientas), el equipo puede trabajar sin trabas y con más motivación.

3. Enunciado claro de los requerimientos (9.6%)

Cuando los requerimientos están bien definidos y claros desde el inicio, el equipo sabe exactamente qué construir. Eso reduce malentendidos y retrabajos

4. Planeamiento adecuado (8.2%)

Un buen plan no solo es tener fechas, sino tener un plan realista, con tareas bien distribuidas, riesgos considerados y objetivos claros. El planeamiento incluye

definir sprints manejables, identificar dependencias, y asegurarse de que el equipo tenga todo lo necesario para cumplir con el plan.

5. **Expectativas realistas (7.7%)**

Esto significa que tanto el cliente como el equipo deben tener expectativas razonables sobre lo que se puede lograr en un tiempo dado. No se trata de prometer lo imposible, sino de alinear las expectativas con la realidad del proyecto.

Si se sabe que una funcionalidad compleja llevará tres sprints y no uno, se comunica de antemano. Así se evitan frustraciones y presión innecesaria.

6. **Hitos intermedios (7.7%)**

Tener hitos intermedios significa definir **pequeños logros** o **entregas parciales** a lo largo del proyecto. **Esto ayuda a medir el progreso, ajustar el rumbo si es necesario, y celebrar pequeños éxitos.**

7. **Personas involucradas competentes (7.2%)**

Esto significa que el equipo de trabajo está formado por personas con las habilidades, el conocimiento y la motivación adecuados para llevar a cabo el proyecto. **En otras palabras, tener un equipo competente y autoorganizado (como se menciona en Scrum) aumenta las probabilidades de éxito.**

-
- **Involucramiento del usuario:** El cliente participa y colabora.
 - **Apoyo de la gerencia:** Recursos y respaldo desde la dirección.
 - **Requerimientos claros:** Sabe exactamente qué se necesita.
 - **Planeamiento adecuado:** Un plan realista y bien gestionado.
 - **Expectativas realistas:** Todos tienen claro qué es posible lograr.
 - **Hitos intermedios:** Metas parciales para medir el progreso.
 - **Personas competentes:** Un equipo capacitado, motivado y autoorganizado.
-

Ingeniería de Software

Parmas definió como "multi-person construction of multi-version software"

Disciplinas de la Ingeniería de Software

Disciplinas Técnicas

La disciplina técnica se refiere a todas aquellas actividades y conocimientos directamente relacionados con la construcción del software. Es decir, todo lo que implica el desarrollo en sí: desde escribir código, diseñar la arquitectura, realizar pruebas, hasta asegurarse de que el software funcione correctamente.

Hablas del “cómo se construye” el software.

Disciplinas Gestión

La disciplina de gestión se enfoca en la planificación, organización, monitoreo y control de los proyectos de software. Es decir, cómo se gestiona el tiempo, los recursos, los riesgos, y cómo se asegura que el proyecto se complete de manera exitosa. Esto incluye actividades como la planificación del proyecto, la gestión de los equipos, la estimación de costos y tiempos, y el seguimiento del avance.

Es la parte que se ocupa del “cómo se organiza y se dirige” el proceso de desarrollo.

Disciplinas de Soporte

Las disciplinas de soporte son aquellas que brindan apoyo transversal a todo el proceso de desarrollo. No están enfocadas directamente en programar o gestionar, sino en asegurar que todo se haga con calidad, orden y buenas prácticas. Esto incluye, por ejemplo, el aseguramiento de la calidad, la gestión de la configuración, y el uso de métricas para medir y mejorar el proceso.

Son las actividades que “soportan” y mejoran el proceso para que se mantenga en el tiempo con la mejor calidad posible.

-
- **Disciplina técnica:** Es el “cómo se construye” el software, las actividades de desarrollo en sí.
 - **Disciplina de gestión:** Es el “cómo se organiza” el proyecto, la planificación y control.
 - **Disciplina de soporte:** Es el “cómo se asegura” la calidad y el orden a lo largo de todo el proceso.
-



Prueba (Testing):

- **Qué es:** La prueba es la disciplina que se encarga de verificar que el software funciona como se espera y cumple con los requisitos. Incluye pruebas unitarias, de integración, de sistema, de aceptación, etc.

Planificación de Proyecto:

- **Qué es:** Es la disciplina que se ocupa de definir los objetivos, alcances, cronogramas, recursos y tareas necesarias para completar el proyecto de software.

Monitoreo y Control de Proyecto:

- **Qué es:** Es la disciplina encargada de supervisar el progreso del proyecto en tiempo real, comparando lo planeado con lo que realmente se está haciendo, e implementando correcciones si es necesario.

Gestión de la Configuración de Software (Configuration Management):

- **Qué es:** Es la disciplina que se encarga de controlar y administrar los cambios en el software, asegurando que todas las versiones y componentes estén bien documentados y gestionados.

Aseguramiento de la Calidad (Quality Assurance):

- **Qué es:** El aseguramiento de la calidad, o QA, se centra en los procesos y actividades diseñadas para que el software cumpla con los estándares de calidad acordados. No se trata solo de probar el software, sino de asegurar que todo el proceso de desarrollo siga buenas prácticas para evitar defectos desde el principio.

Métricas:

- **Qué es:** Las métricas son medidas cuantitativas que se utilizan para evaluar diversos aspectos del proceso y del producto de software. Pueden medir desde la complejidad del código hasta la cantidad de defectos, el rendimiento del equipo, o la satisfacción del cliente.

El proceso de Software

En el lado izquierdo tienes las entradas, que son los insumos iniciales. En el medio, tienes las actividades de trabajo, que es donde realmente se construye el software. Y en el lado derecho tienes las salidas, que son los productos y servicios terminados.

1. Entradas: Requisitos, Ideas y Tiempo:

Las entradas son todo lo que necesitas al inicio del proyecto. Hablamos de los requerimientos, las ideas que el equipo tiene para resolver esos requerimientos, y también el tiempo y otros recursos disponibles.

2. Actividades de Trabajo:

Donde realmente sucede el desarrollo, Las actividades de trabajo incluyen a las personas que desarrollan el software, los materiales o herramientas que utilizan, la energía y el tiempo invertidos, el equipamiento (computadoras, servidores, etc.), y los procedimientos o metodologías que siguen.

3. Salidas: Productos y Servicios

Al final del proceso, obtienes las salidas, que son el software terminado y los servicios asociados. *Esto puede ser una aplicación, un sistema completo, o incluso documentación y soporte.*

✓ Éxito

Definición de Proceso según la IEEE

Secuencia de pasos ejecutados para un propósito dado.

- Esta es una definición general de la palabra proceso, aplicable a cualquier disciplina, no solo software. *Hay un objetivo, conjunto de pasos definidos y un orden lógico para ejecutarlos*

Proceso de Software

Conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

- Esa definición es específica para el mundo del software. Amplía mucho más la idea de proceso. *No son solo pasos lineales, involucra actividades, usa métodos, incluye prácticas y menciona transformaciones, porque pasamos de ideas a código ejecutable.*

Proceso de Software

- Que el proceso de software **no es solo escribir código**.
- Que incluye toda la organización del trabajo para producir un sistema completo y mantenible.
- Que es **sistémico**, es decir, involucra personas, herramientas, procedimientos y más.

Personas con habilidades, entrenamiento y motivación

Las personas son el **factor central** de cualquier proceso de software.
Sin desarrolladores capacitados y motivados, no hay proceso que funcione.

- El software es un trabajo **intelectual y creativo**, por eso las personas no son reemplazables como en una línea de ensamblaje.
- La calidad final depende mucho del **nivel técnico y actitud del equipo**.
- Es importante la **formación continua** (entrenamiento) y el **compromiso** (motivación).

Procedimientos y métodos

Son las formas estandarizadas de hacer las cosas.

Método: es un enfoque o estrategia general (ej. método ágil, método en cascada).

Procedimiento: son los pasos detallados que indican cómo aplicar ese método en la práctica.

Herramientas y equipos

Es todo lo **tecnológico y físico** que se usa para desarrollar el software.

- Las herramientas **soportan** el proceso, pero no lo garantizan.
- Elegir las herramientas correctas **mejora la productividad** del equipo.
- Un buen equipo sin herramientas puede trabajar, pero más lento; un equipo sin competencias ni motivación, ni con las mejores herramientas puede entregar buen software.

Proceso

El proceso de software es la combinación ordenada y coordinada de:

- Personas competentes
- Procedimientos/métodos apropiados

- Herramientas adecuadas

Procesos definidos vs procesos empíricos

Proceso definido

Un proceso definido es aquel en el que las actividades están claramente especificadas, estandarizadas y controladas. *Se caracteriza porque: Si se repite bajo las mismas condiciones, se espera obtener los mismos resultados.*

- Este enfoque se basa en la predictibilidad y la repetibilidad. Es típico de entornos como la manufactura tradicional, donde los procesos son medibles, predecibles y estables.

Proceso empírico

Un proceso empírico es aquel en el que no se puede predecir completamente el resultado de antemano, debido a que el entorno es complejo, cambiante o incierto, En lugar de basarse en predicción, se basa en la observación, la adaptación y control mediante retroalimentación. Este es el enfoque que utiliza Scrum.

- Alta variabilidad e incertidumbre
- Cambios frecuentes en requerimientos
- Resultados emergentes
- Se requiere feedback constante
- Basado en ciclos cortos de inspección y adaptación

Característica	 Proceso Definido	 Proceso Empírico
Predictibilidad	Alta (se puede prever el resultado)	Baja (resultado cambia según contexto)
Variabilidad	Mínima	Alta
Control basado en...	Planificación, estándares y procedimientos	Inspección, adaptación y feedback continuo
Ejemplo típico	Fabricación de autos	Desarrollo ágil de software
Metodologías que lo aplican	Cascada, RUP, procesos ISO	Scrum, XP, Kanban
Relación entradas-salidas	Entradas iguales → salidas iguales (en teoría)	No se puede garantizar, se ajusta en el camino
Requerimientos	Estables, bien definidos al inicio	Cambiantes, se descubren con el cliente

Ciclos de vida

La serie de pasos a través de los cuales el producto o proyecto progresa. Los productos tienen su ciclo de vida. Los proyectos también.

El ciclo de vida es el conjunto de fases o etapas por las cuales pasa un producto o un proyecto desde que nace hasta que finaliza.

Aplica tanto a:

- Productos de software
- Proyectos de Software

Ciclo de Vida	¿Qué abarca?	¿Cuándo empieza y termina?	En qué se enfoca
Producto	Todo el tiempo que el producto existe.	Desde la idea inicial hasta el retiro del sistema .	En el uso y evolución del producto.
Proyecto	El esfuerzo limitado en el tiempo para crear o actualizar un producto.	Desde la planificación del proyecto hasta la entrega (fase operativa o despliegue).	En el trabajo necesario para construir o actualizar el producto.

Ejemplo

- **Ciclo de vida del producto:** una app bancaria desde su concepción (2015), pasando por versiones, actualizaciones, soporte... hasta que se deja de usar (2025).
- **Ciclo de vida del proyecto:** los distintos esfuerzos organizados para crear la versión 1.0, luego la 2.0, una migración, etc.

Éxito

¿Por qué es importante diferenciarlos?

- Porque el **producto** puede tener muchos **proyectos** a lo largo de su vida.
- Porque **las decisiones de gestión, recursos y planificación** son distintas si pensás a largo plazo (producto) o corto plazo (proyecto).

"Un ciclo de vida de un proyecto de software es una representación de un proceso"

Grafica y organiza cómo se estructura el proceso de desarrollo desde un punto de vista práctico.

Los modelos de ciclo de vida indican:

- Las **fases** (requerimientos, análisis, diseño, desarrollo, pruebas, etc.).
- El **orden** en que se llevan a cabo.

Tipos de ciclo de vida

1. Secuencial

El más tradicional, cada fase se ejecuta una sola vez y en orden, muy estructurado, poco flexible, ideal cuando los requerimientos están bien definidos desde el principio. Difícil de manejar los cambios.

2. Iterativo/Incremental

Desarrollas el producto por partes o versiones, repitiendo ciclos cortos. Cada iteración agrega valor (funcionalidad). Los requerimientos pueden cambiar y adaptarse entre iteraciones. Scrum usa este enfoque. Se adapta mejor al cambio, entrega valor desde temprano. Requiere más gestión y retroalimentación continua.

3. Recursivo

Similar al iterativo, pero con más enfoque en refinamiento continuo. Hay actividades que se repiten dentro de cada fase, no solo entre versiones. Se aplica a procesos de mejora continua o exploratorios. Permite revisión y corrección constante. Puede volverse caótico si no se controla bien.

¿Qué relación hay entre ciclos de vida y procesos?

Ciclo de vida: Te dice qué actividades tenés que hacer

Proceso de desarrollo: Te dice cómo vas a hacer esas actividades

Consejo

En resumen:

- **Ciclo de vida** = qué fases hay.
- **Proceso** = cómo vas a trabajar en esas fases.

Resumen para examen

¿Qué es un ciclo de vida?

Es una representación de las fases que atraviesa un producto o proyecto a lo largo del tiempo.

¿Para qué sirve?

Para organizar el trabajo, planificar, controlar el avance y asegurar la calidad del producto final.

¿Qué tipos existen?

Tipo	Explicación breve	Ejemplo
Secuencial	Fases en orden, sin volver atrás	Cascada
Iterativo / Incremental	Se repiten ciclos cortos agregando valor	Scrum
Rekursivo	Actividades que se refinan dentro de cada fase	Mejora continua

Diferencia entre ciclo de vida de producto y proyecto:

Ciclo de vida	Enfocado en...	Dura...
Producto	Uso del sistema a largo plazo	Años
Proyecto	Desarrollo o actualización puntual	Meses o semanas

SCM Gestión de la Configuración de Software

Es una disciplina de soporte dentro de la Ingeniería de software que se encarga de controlar y administrar los cambios que se hacen sobre los artefactos del software durante su ciclo de vida.

Es el conjunto de prácticas que asegura que sepamos qué se cambió, cuándo, por qué, por quién y cómo dentro de un proyecto de software

Objetivos clave

- Controlar las distintas versiones del software
- Evitar conflictos entre desarrolladores que trabajan en paralelo
- Poder volver a versiones anteriores ante alguna falla
- Mantener la trazabilidad entre requisitos, código, documentación y pruebas
- Asegurar la integridad del producto durante su evolución

SCM forma parte de las disciplinas de soporte, junto con el Aseguramiento de calidad, Medición y métricas y Documentación técnica.

Lo que diferencia estas disciplinas, es que **no codifican directamente, no gestionan personas o proyectos, pero garantizan que todo funcione bien a largo plazo**

Tipo de proceso	Rol del SCM
Definido	El SCM es parte fundamental del control . Se usa para asegurar que cada paso del proceso esté documentado, que no haya desviaciones, y que todo cambio sea aprobado formalmente.
Empírico	El SCM es igualmente esencial , pero con un enfoque más ágil. Se adapta a cambios frecuentes y rápidos. Ayuda a mantener el orden en proyectos donde hay muchas iteraciones e incrementos.

En Scrum, aunque se trabaje con iteraciones cortas y flexibilidad, se necesita sí o sí:

- **Control de versiones (Git)**
- **Ramas de trabajo**
- **Revisión de cambios (pull requests)**
- **Historial de commits**

Todo eso es parte del SCM, incluso si no se lo menciona directamente con ese nombre.

¿Qué elementos se gestionan en el SCM?

- **Código fuente**
- **Documentación (manuales, especificaciones)**
- **Casos de prueba**
- **Archivos de configuración**
- **Requisitos y cambios de requerimientos**

Componentes típicos del SCM

1. **Control de versiones:**
Ej.: Git, Subversion, Mercurial.
→ Registra los cambios y permite trabajar en paralelo.
2. **Gestión de cambios:**
→ Evalúa, aprueba o rechaza propuestas de cambio.
3. **Gestión de la línea base (baseline):**
→ Define momentos "congelados" del producto para control.
4. **Auditoría y seguimiento:**
→ Verifica que lo que se construyó corresponde con lo planificado.

SCM es la disciplina de soporte que **controla los cambios** sobre todos los elementos del software (código, requisitos, documentación) a lo largo de su ciclo de vida.

Se utiliza tanto en procesos **definidos** (para garantizar trazabilidad y control) como en **empíricos** (para mantener el orden ante cambios frecuentes).

Su implementación típica incluye **control de versiones**, **gestión de cambios**, y **auditorías**. Es esencial para proyectos colaborativos y productos en evolución.

- Que es una **disciplina de soporte** clave en la Ingeniería de Software.
- Que **permite gestionar los cambios** y versiones del software.
- Que es útil en **procesos definidos y empíricos**, aunque se use distinto.
- Que **garantiza trazabilidad, orden y control**, especialmente en equipos grandes.
- Que no se limita solo al código: incluye todos los artefactos del proyecto.

Software en contexto



PROCESO

- Es el conjunto estructurado de **actividades, prácticas y métodos** que se usan para desarrollar el software.
- El proceso **no garantiza la calidad por sí solo**, pero **define cómo se trabaja**.
- Puede ser un proceso **definido** (planificado y controlado) o **empírico** (basado en inspección y adaptación).

“¿Qué tiene que tener un software para que sea íntegro? Que tenga valor para el cliente...

Nada garantiza la calidad.”

Esto refleja una idea central: **el proceso bien definido ayuda, pero no alcanza**. La calidad depende de cómo se aplica el proceso, de las personas, de las herramientas, etc.

PROYECTO

- Es el esfuerzo temporal para construir o mejorar un producto.
- El proceso se **aplica al proyecto**.

Es decir:

- Si el proceso está definido de antemano (como en una empresa), el proyecto **se adapta** a ese proceso.
- Pero en equipos más pequeños o ágiles, el proceso **se puede adaptar** al proyecto (por ejemplo, si es un desarrollo muy exploratorio).

PERSONAS

- Las personas **se incorporan al proyecto** y son quienes ejecutan el proceso.
- Aportan **conocimiento, experiencia, habilidades y juicio**.
- Sin personas capacitadas y motivadas, ningún proceso ni herramienta sirve.
- En metodologías ágiles, se da **más peso a las personas** que a los procesos (como dice el Manifiesto Ágil).

Información

Manifiesto Ágil

Individuos e interacciones sobre procesos y herramientas
Software funcionando sobre documentación extensiva
Colaboración con el cliente sobre negociación contractual
Respuesta ante el cambio sobre seguir un plan

Esto es, aunque valoramos los elementos de la derecha,
valoramos más los de la izquierda.

HERRAMIENTAS

- Sirven para **automatizar partes del proceso** y facilitar el trabajo.

“Automatizado con herramientas (la gestión de la configuración del software, por ejemplo).”

Esto te recuerda que **SCM** es una disciplina que se **apoya fuertemente en herramientas**. Pero también en prácticas y procedimientos (no es solo tecnología).

PRODUCTO

- Es el **resultado** del proyecto.
- No es solo el software ejecutable: también incluye documentación, configuración, pruebas, etc.
- Debe tener valor, ser funcional, confiable y mantenible.

Elemento	¿Qué aporta?	¿A quién se conecta?
Personas	Conocimiento, creatividad, ejecución	Ejecutan el proceso dentro del proyecto
Proceso	Organización, disciplina, estructura	Se aplica al proyecto y es soportado por herramientas
Proyecto	Contexto de ejecución (tiempo, recursos, metas)	Aplica el proceso, incluye personas, usa herramientas
Herramientas	Automatización, soporte técnico	Facilitan el proceso y lo hacen más eficiente
Producto	Resultado final	Es el objetivo del proyecto y resultado del proceso

Información

Rol de la Gestión de la Configuración del Software (SCM)

En tu anotación decís:

“Disciplina de soporte protectora. Se usa para mantener la integridad del software.”

 **SCM es la disciplina que se asegura de que el producto no pierda coherencia a lo largo del tiempo.**

 Lo hace mediante:

- Control de versiones
- Control de cambios
- Líneas base
- Identificación de configuración

 Esto es crítico porque el software es **malleable y cambia constantemente**. Entonces SCM es lo que permite que esos cambios **no rompan la estabilidad ni la calidad**.

Cuando pensamos en Software... en qué pensamos?

Un error común es pensar que el software es solo el código fuente.

Esto es incompleto y peligroso. El código es solo una parte del software.

El software es un conjunto de elementos interrelacionados que permiten que un sistema funcione como un todo. No solo incluye programas, sino la lógica, las reglas, datos y toda la documentación asociada.

Software = conjunto de Programas + Procedimientos + Reglas + Documentación + Datos

Programas

Archivos ejecutables y el código fuente que realiza funciones específicas

Procedimientos

Son los pasos o métodos prácticos que deben seguirse para usar, instalar, ejecutar o mantener el software.

Reglas

Normas del negocio, restricciones y validaciones que el software implementa.

Documentación

Incluye todos los materiales escritos o gráficos que explican el uso, diseño, arquitectura, instalación, mantenimiento y pruebas del software

Datos

Todo software necesita información para operar, puede ser de entrada, de salida o configuración. Estos también forman parte del software como producto.

Cambios en el software

Causa del cambio	Ejemplo real
Cambios en el negocio	Nueva legislación obliga a modificar el sistema contable.
Nuevos requerimientos del cliente	El cliente quiere soporte para móvil en la próxima versión.
Cambio en productos relacionados	La base de datos o el sistema operativo cambian.
Reorganización interna	Se redefine el presupuesto y se achica el equipo.

Causa del cambio	Ejemplo real
Defectos encontrados	Aparecen errores funcionales luego de la entrega.
Oportunidades de mejora	Se detecta una forma más eficiente de hacer un proceso.

El software **nunca es estático**. Siempre está **evolucionando**.

Qué rol cumple SCM frente a los cambios

El SCM es la disciplina que controla y protege el producto de software frente a los cambios

Funciones clave:

1. **Registrar cada cambio.**
2. **Trazar su origen.**
3. **Evaluar el impacto.**
4. **Aprobar o rechazar.**
5. **Implementar con control.**
6. **Versionar el producto resultante.**

Esto garantiza que el software siga siendo:

- Íntegro
- Estable
- Rastreable
- Profesional

¿Por qué se dice que SCM es una **disciplina paraguas**?

La **idea de "disciplina paraguas"** significa que **cubre o atraviesa** todas las fases del proyecto de software.

No es una tarea que se haga solo en una etapa, sino que está **presente a lo largo de todo el ciclo de vida** del software.

¿Cuándo actúa SCM?

- Durante el análisis de requisitos: guarda versiones.
- Durante el diseño: controla cambios de especificaciones.
- Durante la codificación: controla versiones del código.
- Durante las pruebas: mantiene la trazabilidad entre pruebas y requisitos.
- Durante el mantenimiento: gestiona actualizaciones.

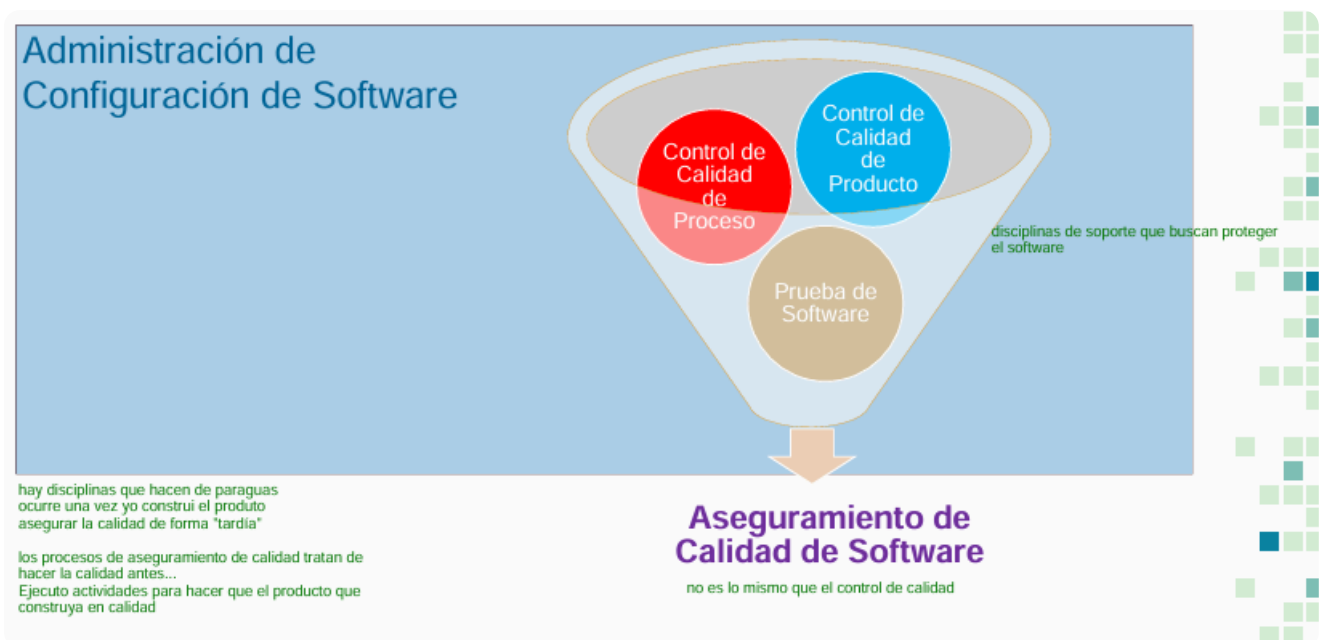
Ejemplo

Cuando un error se reporta en producción, gracias al SCM podés:

- Ver qué versión lo introdujo.
- Saber qué desarrollador lo cambió.
- Saber qué requisito afectó.
- Corregirlo sin romper todo lo demás.

SCM como **disciplina de soporte y protección**

- Es **de soporte** porque **no desarrolla funcionalidad**, pero **habilita y protege** todo lo que se construye.
- Es **de protección** porque:
 - Evita inconsistencias.
 - Protege de errores humanos.
 - Permite auditar y recuperar estados anteriores.



La imagen tiene un embudo, que representa cómo distintas disciplinas alimentan el aseguramiento de la calidad.

1. **Administración de Configuración de Software (SCM)**

Está por fuera del embudo porque **atraviesa todo**. Es la disciplina **paraguas** que acompaña todo el ciclo.

2. **Control de Calidad del Proceso**

Evalúa si el proceso seguido fue el adecuado.

Ej: ¿Se siguieron las prácticas de revisión de código?

3. Control de Calidad del Producto

Evalúa el producto entregado.

Ej: ¿La funcionalidad cumple con los requisitos?

4. Pruebas de Software

Detecta errores en el funcionamiento.

Es una actividad de verificación directa del producto.

✓ Éxito

🎯 Aseguramiento de Calidad $\neq$ Control de Calidad

📌 **Aseguramiento de la calidad** = Prevenir errores

📌 **Control de calidad** = Detectar errores

El SCM es **una disciplina de soporte, paraguas y protectora** del software.

Permite gestionar los **cambios inevitables** en el producto de manera ordenada y profesional.

Se relaciona con el aseguramiento de calidad y con el proceso completo de desarrollo.

Definición de SCM por ANSI/ IEEE 828, 1990

Una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos.

¿Por qué es necesario gestionar la configuración del software?

La **gestión de la configuración del software** es una **disciplina de soporte** que tiene como **propósito establecer y mantener la integridad** de los productos de software a lo largo de su ciclo de vida.

- Que el software siga siendo **coherente, rastreable y funcional** a medida que cambia.
- Que **sepa qué cambió, cuándo cambió, por qué y quién lo cambió**.
- Que **pueda reconstruirse el producto exacto** a partir de sus componentes.

Riesgos si no se gestiona bien

Riesgo / Problema	Qué significa
 Pérdida de un componente	Archivos fuente o configuración desaparecen.
 Regresión de fallas	Algo que funcionaba ahora vuelve a fallar.
 Cambios no validados	Alguien hizo un cambio que no se aprobó ni se testeó.
 Doble mantenimiento	Dos personas mantienen lo mismo sin saberlo, duplicando trabajo.
 Superposición de cambios	Dos desarrolladores modifican la misma parte y se pisan.
 Sincronía fuente-objeto	El ejecutable no coincide con el código fuente.
 Pérdida de cambios	Se sobrescriben o borran versiones anteriores.

¿Qué logra la gestión de configuración?

Objetivo Clave	Qué significa
 Identificación de la configuración	Saber exactamente qué elementos componen cada versión del software.
 Control de cambios	Asegurar que todo cambio sea autorizado, controlado y trazable .
 Control de versiones	Poder acceder a versiones anteriores, comparar, recuperar.
 Auditoría y registro	Saber quién hizo qué y cuándo .
 Trazabilidad	Relacionar requerimientos ↔ código ↔ pruebas ↔ entregas.
 Mantenimiento de la integridad	Garantizar que el producto final sea coherente y válido .

¿Qué significa "**integridad del producto**"?

Criterio	Ejemplo
 Satisface necesidades del usuario	Cumple los requerimientos funcionales y no funcionales.
 Es completamente trazable	Se puede rastrear cada requerimiento hasta su código y prueba.
 Cumple criterios de performance	No se rompe al escalar, cumple tiempos de respuesta.

Criterio	Ejemplo
 Cumple expectativas de costo	Los cambios no generan retrabajo innecesario ni demoras.

SCM es esencial durante TODO el ciclo de vida del producto (no solo durante el proyecto).

Aunque SCM es muy poderosa, **no hace magia**. No sirve para:

No sirve para...	Porque...
Detectar errores funcionales	Eso lo hace el testing, no SCM.
Controlar costos directamente	Pero sí evita costos ocultos por desorganización.
Reemplazar el diseño o la arquitectura	No es un reemplazo de buenas decisiones técnicas.
Suplir comunicación entre personas	No reemplaza reuniones, aclaraciones ni planificación colaborativa.
Detectar errores funcionales	Eso lo hace el testing, no SCM.
Controlar costos directamente	Pero sí evita costos ocultos por desorganización.
Reemplazar el diseño o la arquitectura	No es un reemplazo de buenas decisiones técnicas.
Suplir comunicación entre personas	No reemplaza reuniones, aclaraciones ni planificación colaborativa.

Conceptos clave de Gestión de la Configuración de Software

- **Ítem de Configuración**
- **Repositorio**
- **Línea Base**
- **Ramas**
- **Configuración del Software**

Ítem de configuración

Cita

"Un ítem de configuración es cualquier artefacto que debe ser gestionado, mediante control de configuración. Puede ser un archivo, un documento, un código fuente, un ejecutable, un requerimiento, una imagen, etc"

→ [The Little Book of CM](#)

- Tiene un nombre único e inmutable
- Son versionables
- Son trazables
- Forman el producto o el proyecto

Ejemplos: Código fuente, Documentación, Scripts de la base de dato, Diagramas UML o de la arquitectura, Casos de prueba, Instaladores.

Son la unidad básica de gestión. Todo el control de versiones, trazabilidad y auditoría gira en torno a ellos

Repositorio

” Cita

"Un repositorio es el entorno que almacena y gestiona los ítems de configuración y sus versiones a lo largo del tiempo"

→ [SCM Agile - Bersoff](#)

Tipos:

- **Centralizado:** Un servidor único
- **Distribuido:** Cada desarrollador tiene una copia completa

Funciones:

- Almacena versiones históricas
- Controla accesos
- Permite comparar, extraer, actualizar y publicar
- Da soporte al trabajo colaborativo

Línea Base

” Cita

"Una línea base es una configuración específica de ítems (con versiones especificadas) que ha sido formalmente revisada y acordada, y sirve como base para desarrollos futuros."

→ [Pressman, Ingeniería de Software](#)

- Es como una fotografía del proyecto en un momento específico
- Se utiliza para referenciar un estado estable

- Toda modificación sobre la línea base debe pasar por un proceso formal de control de cambios

Ejemplos:

- Línea base de requisitos
- Línea base de diseño
- Línea base del producto

Se utilizan tags para marcar líneas base. Se marca una línea base porque al terminar una etapa crítica del proyecto, se puede empezar a continuar con una base estable

Configuración del software

” Cita

"La configuración del software es el conjunto de ítems de configuración y sus versiones asociadas en un momento determinado del tiempo.

→ [The little book of CM](#)"

Una foto del proyecto: qué versión de cada archivo existía en ese instante

No toda configuración es estable o íntegra. Algunas son transitorias o incompletas, especialmente durante el desarrollo.

Versión vs Variante

” Cita

"Una versión es una modificación de un ítem de configuración. Representa su evolución en el tiempo."

→ [SCM Agile - Bersoff](#)

El control de versiones se refiere a la evolución de un único ítem de configuración o de cada ítem por separado

” Cita

"Una variante es una versión de un ítem de configuración que se desarrolla en paralelo para un contexto distinto: plataforma, cliente, entorno"

→ [The little Book of CM](#)

Un producto de software puede adoptar distintas formas dependiendo del lugar que se instale

Concepto	Versión	Variante
¿Qué cambia?	Evolución del mismo CI	Adaptación para un contexto diferente
¿Cómo se gestiona?	En la misma línea temporal	En ramas distintas, a veces paralelas
Ejemplo	<code>login_v1</code> → <code>login_v2</code>	<code>login_win</code> vs <code>login_linux</code>

Ramas

” Cita

"Una rama es una bifurcación en la evolución de un conjunto de ítems. Se usa para desarrollar nuevas funcionalidades sin afectar la línea principal."

→ [SCM AGILE - Bersoff](#)

- Permiten trabajo paralelo
- Usadas para experimentar, corregir bugs, desarrollar versiones específicas
- Pueden fusionarse o descartarse
- Cada rama puede tener su propia línea base

Concepto	Qué es	Ejemplo
Ítem de configuración	Artefacto versionable y trazable	<code>requisitos.docx</code>
Repositorio	Sistema que guarda y gestiona los ítems	GitHub
Configuración	Conjunto de ítems + sus versiones en un momento dado	Snapshot del proyecto
Línea base	Configuración estable y aprobada	<code>release 1.0</code>
Versión	Evolución temporal de un ítem	<code>login.py v2.1</code>
Variante	Configuración para otro entorno/contexto	<code>login_android</code>
Rama	Desvío de desarrollo	<code>feature/registro</code>

4 Actividades Fundamentales de la Gestión de Configuración

1. Identificación de Ítems de Configuración

” Cita

"La identificación de configuración es el proceso de seleccionar, nombrar, y documentar los ítems de configuración que van a ser controlados"

"La capacidad de de identificar qué información ha sido aprobada para ser utilizada de forma concurrente en el proyecto, quién es el responsable de esa información, cómo fue aprobada para quedar bajo control de configuración, y cuál es la última versión aprobada"

→ [The little Book of Configuration Managment](#)

- Establecer qué ítems van a ser gestionados, cómo se llaman, y cómo se agrupan
- Asegurar que cada ítem tenga un nombre único y persistente, para poder rastrearlo en el tiempo
- Define **qué ítems forman parte de la configuración** (documentos, código, pruebas, etc.).
- Asigna a cada uno un **nombre único**, versión y responsable.
- Establece las reglas de nombrado y clasificación (por ejemplo: **REQ-001-v1.0**).
- Es el punto de partida: sin esta identificación, **no hay control posible**.
- Ubicación dentro de la estructura de repositorio

2. Control de cambios

” Cita

Control de configuración es el proceso mediante el cual se define el nivel de control que debe tener cada producto de trabajo. Incluye procedimientos para solicitar, evaluar, autorizar, aplicar y rastrear los cambios. El control de cambios permite que el sistema pueda evolucionar ordenadamente y sea reproducible.

→ [The little Book of Configuration Managment](#)

- Define quién puede aprobar un cambio
- Establece los pasos para realizar un cambio: solicitud, evaluación, aprobación, implementación
- Permite saber qué se cambió, cuándo, por qué y quién lo hizo
- Sin esto, los cambios serías caóticos y no podrían replicarse ni controlarse
- Surge con un requerimiento de cambio a uno o varios items de configuración que se encuentran en la línea base. Es el procedimiento formal que involucra diferentes actores y una evaluación del impacto del cambio

- Existe la presencia del comité de cambios, conformado por representantes de las áreas involucradas del desarrollo.

3. Informes de estado

” Cita

Informe de estado de configuración es la actividad que formaliza el registro y el reporte de los documentos establecidos, del estado de los cambios propuestos y del estado de la implementación de los cambios aprobados.

→ [*The little Book of Configuration Managment*](#)

- Mantiene un registro accesible y actualizado del estado de cada ítem de configuración
- Informa qué cambios están en curso, aprobados, rechazados o aplicados
- Brinda trazabilidad y visibilidad del estado del proyecto a los stakeholders
- Es esencial para auditorías, releases y gestiones del proyecto

4. Revisiones y Auditorías

” Cita

Revisiones y auditorías consisten en evaluaciones frecuentes del contenido, de la integridad de la línea base y de la integridad de las entregas de todos los productos controlados, para asegurar que cumplen con su documentación de configuración.

→ [*The little Book of Configuration Managment*](#)

- Validan que los productos del proyecto estén completos, coherentes y aprobados
- Verifican que lo que se entrega cumple con lo documentado
- Pueden ser funcionales (cumple con lo que dice) o físicas (están los archivos)
- Se aplican antes de releases, entregas a cliente o hitos críticos

Plan de Gestión de Configuración

El **SCM Plan** es un documento **estratégico y técnico** que define cómo se llevará a cabo la gestión de configuración durante el proyecto. Es **parte esencial de la planificación inicial** y se desarrolla de forma paralela al Plan de Desarrollo del Software.

¿Qué debería incluir el plan?

- ❖ Reglas de nombrado de los CI

- ❖ Herramientas a utilizar para SCM
- ❖ Roles e integrantes del Comité
- ❖ Procedimiento formal de cambios
- ❖ Plantillas de formularios
- ❖ Procesos de Auditoría

1. Introducción

- Propósito del plan.
- Alcance (qué se cubre y qué no).
- Aplicabilidad (a qué proyectos, equipos o versiones aplica).

2. Organización del proyecto

- Describe cómo se relaciona el área de Gestión de Configuración con otras áreas (Calidad, Ingeniería, Gestión, Cliente).
- Cómo se comunica, interactúa y se alinea con los objetivos generales.
En otras palabras: cómo **SCM no es un compartimento aislado**, sino parte integral del proyecto.

3. Fases del proyecto y Hitos

- Identifica los **momentos clave** donde se establecerán **líneas base formales**.
- Establece la **evolución temporal del SCM** en el ciclo de vida del software.

4. Identificación de Configuración

- Lista qué **artefactos** estarán bajo control (requisitos, documentos, código, casos de prueba, scripts, etc.).
- Define las **reglas de nombrado**, versiones, y organización de carpetas.
- Define las **convenciones** de numeración, codificación y clasificación.

5. Gestión de Interfaces

- Describe cómo se identifican, documentan y gestionan **interfaces internas y externas**.
- Define el procedimiento para llegar a acuerdos y realizar **control de cambios sobre interfaces**.

6. Control de Cambios (Change Control)

- Define los **procedimientos para solicitar, analizar, aprobar, implementar y verificar cambios**.
- Determina los **niveles de autorización** para realizar cambios según el impacto.
- Detalla cómo se gestiona el **comité de cambios (CCB – Change Control Board)**.

7. Contabilidad de Estado (Status Accounting)

- Define cómo se registran y reportan:
 - El estado actual de los ítems de configuración.
 - El estado de cambios propuestos y aprobados.
 - Qué versión está liberada y cuál en desarrollo.

8. Auditorías de Configuración

- Define los procesos para:
 - Verificar que el producto final coincide con su configuración.
 - Validar que los cambios fueron correctamente aplicados.
- Determina **frecuencia**, responsables y procedimientos.

9. Gestión de Subcontratistas

- Procedimientos para verificar que **proveedores o terceros** cumplen con las normas de configuración.
- Muy importante en grandes proyectos con desarrollos distribuidos.

Reglas(Algunas)

- **Rule 1:** El control debe hacerse *a tiempo*, no cuando ya es tarde.
- **Rule 2:** La identificación y el control de cambios debe hacerse *temprano*.
- **Rule 5:** El cambio debe ser parte de la *cultura del proyecto*, no una molestia burocrática.
- **Rule 10:** Primero el proceso, después la herramienta. No adaptes tu proceso a la herramienta; elegí una herramienta que respete tus procesos.

Gestión de la Configuración en Agile

- **No busca controlar desarrolladores**, sino **coordinar y sincronizar su trabajo**.
- **Sirve al equipo**, no al revés: SCM está al servicio del desarrollo, no lo restringe.
- **Favorece la automatización extrema**, integración frecuente y entregas rápidas.
- **Minimiza fricción**: Promueve trazabilidad, transparencia, visibilidad de cambios.
- Usa **repositorios distribuidos, CI/CD, scripts de build y automatización** de merges.

Qué pasa con el comité de cambios en Agil

- En entornos tradicionales, existe un **Change Control Board (CCB)** o comité que aprueba formalmente los cambios.
- En Ágil, esta figura **se vuelve más liviana o incluso desaparece**.

- La **autoridad de cambio está más distribuida**: cada desarrollador tiene autonomía (dentro de su rama) y los cambios se integran **rápidamente** mediante integración continua y revisiones automatizadas.

Rule 3:

“El proceso de gestión de cambios debe ser simple, coherente con la cultura y los requerimientos del proyecto, y estar respaldado por los métodos y herramientas del mismo.”

Esta regla **desaconseja el control rígido** e **impulsa un enfoque adaptado al entorno de desarrollo**, tal como sucede en metodologías ágiles.

Qué ítem hay que mantener

En ágil, **los ítems de configuración (CI)** no cambian en su esencia:

- Código fuente
- Casos de prueba
- Documentación útil (como scripts de build, archivos README)
- Archivos de configuración del entorno (Dockerfiles, .env, CI config, etc.)
- Releases etiquetadas

La diferencia es que **todo se versiona, pero solo lo esencial se gestiona** activamente. Se evitan ítems innecesarios o de bajo valor.

” Cita

“Describe your build processes using scripts rather than procedures. SCM can automate development policies and processes with executable knowledge.”

— Agile SCM, p.6

¿Qué pasa con las auditorías?

- Las auditorías formales son **reemplazadas o complementadas** por **checks automáticos**, como:
 - Tests automáticos (unitarios, de integración, regresión)
 - Linters, checkers de seguridad, análisis estático
 - Revisiones en pull requests (merge requests)
- No hay "auditorías en papel", sino que **la integridad del software se valida por herramientas** y pipelines.

La gestión de configuración en ágil **no desaparece**, sino que **evoluciona**:

-  Los elementos clave (líneas base, ítems, cambios, auditorías) **siguen existiendo**
-  Se reemplaza lo burocrático por **procesos automatizados y colaborativos**
-  Se reparte la responsabilidad entre todos los miembros del equipo
-  **Las herramientas (Git, Jenkins, GitLab, etc.) reemplazan formularios y reportes manuales**
-  SCM en ágil **sirve para acelerar el desarrollo y garantizar calidad continua**

Definición bien completa SCM

La **Gestión de la Configuración de Software (SCM)** es una disciplina de soporte esencial de la Ingeniería de Software cuyo propósito principal es **establecer, mantener y resguardar la integridad, trazabilidad, estabilidad y control evolutivo** de los productos de software y sus artefactos asociados, **a lo largo de todo su ciclo de vida**.

Su objetivo es asegurar que **cada elemento que compone el software esté identificado, versionado, controlado y verificado**, de modo que cualquier cambio realizado sea **comprensible, rastreable, seguro y coherente con los objetivos del proyecto y del negocio**.

Conceptos clave

Concepto	Definición
Ítems de Configuración (CI)	Todo artefacto del producto o proyecto que pueda cambiar, sea compartido o requiera control (ej: código fuente, documentación, casos de prueba, requisitos, manuales, scripts de build, etc.).
Configuración de Software	El conjunto específico de ítems de configuración con sus respectivas versiones en un momento determinado. Es una “fotografía” del estado del sistema.
Línea base (Baseline)	Una configuración establecida formalmente, aceptada y controlada a partir de la cual se realizan desarrollos posteriores. Representa una versión íntegra, validada.
Versión	Forma evolutiva de un ítem de configuración. Implica modificaciones respecto a versiones anteriores.
Variante	Forma paralela de un ítem o configuración adaptada a contextos específicos (por ejemplo, para distintos S.O.).
Repositorio	Sistema de almacenamiento que mantiene los ítems de configuración, sus versiones y relaciones. Ej: Git, SVN.
Branch (rama)	Línea de desarrollo paralela que permite el trabajo en evoluciones distintas sin afectar la rama principal.
Merge	Integración de los cambios de una rama con otra.

Actividades Fundamentales

Actividad	Qué hace
1. Identificación de ítems	Selecciona, nombra y categoriza los artefactos que deben ser controlados. Aplica convenciones de nombrado y clasificación.
2. Control de cambios	Gestiona y formaliza las solicitudes de modificación. Define roles (como el Comité de Cambios), flujos de aprobación, evaluaciones de impacto y control de versiones.
3. Auditoría de configuración	Verifica que los ítems estén correctamente registrados, sean coherentes entre sí, cumplan con lo establecido, y estén correctamente versionados.
4. Informes de estado	Permite conocer en todo momento el estado y evolución de cada ítem o configuración: quién lo modificó, cuándo, qué versión, en qué estado se encuentra.

Para qué sirve

-  **Trazabilidad** total del software y sus componentes.
-  **Integridad**: garantiza que el software mantiene coherencia entre versiones.
-  **Control del cambio**: evita superposición, pérdida o conflicto de cambios.
-  **Auditoría y transparencia**: hace visible el estado del proyecto en cada instante.
-  **Automatización y coordinación**: permite flujos ágiles de trabajo.
-  **Reducción de riesgos**: minimiza fallas por versiones mal integradas.
-  **Facilidad de reproducción**: reconstrucción exacta de versiones anteriores.

Fases de implementación de SCM

1. Planificación del SCM

- Se define el *Plan de Gestión de Configuración*, que contiene:
 - Objetivos
 - Convenciones de nombrado
 - Herramientas a utilizar
 - Roles (como el Comité de Cambios / Change Control Board)
 - Procedimientos de auditoría y control
 - Plantillas, formularios, métricas y reglas
- Fuente: *The Little Book of Configuration Management*, Best Practice #5

2. Identificación de ítems

3. Control de versiones y cambios

4. Seguimiento y reportes (Status Accounting)

5. Auditorías y verificación
6. Mantenimiento y evolución
7. Cierre de proyecto y archivo de configuraciones

SCM en contextos ágiles

En entornos ágiles

-  SCM se adapta a la velocidad del equipo.
-  Se apoya en automatización (CI/CD).
-  Se distribuyen responsabilidades (no hay un solo dueño).
-  La validación se hace con pruebas automáticas y revisiones.
-  La trazabilidad y documentación están integradas al flujo de trabajo.

Reglas y Colaciones de CM

Regla 1

” Cita

CM must manage the ownership of information and bring work products under control in a timely manner.

✦ *La gestión de configuración debe administrar la propiedad de la información y poner los productos de trabajo bajo control en el momento adecuado.*

SCM no se trata solo de guardar archivos, sino de saber quién es responsable de qué y cuándo un producto pasa a estar bajo control formal.

Regla 2

” Cita

Early identification and change control of artifacts and work products is integral to a project.

✦ *La identificación temprana y el control de cambios de artefactos y productos de trabajo es esencial para un proyecto.*

Cuanto antes se controle un ítem, mejor. Esperar demasiado aumenta el desorden y reduce la trazabilidad

Regla 3

” Cita

The change management process must be simple, consistent with the culture and requirements of the project, and supported by the methods and tools of the project.

✦ *El proceso de gestión de cambios debe ser simple, coherente con la cultura y requerimientos del proyecto, y estar apoyado por herramientas y métodos del proyecto.*

El SCM se adapta al proyecto, no lo revienta con burocracia. Clave en entornos ágiles.

Regla 4

” Cita

The actions, roles, and responsibilities of the Engineering Review Board (ERB) and the Change Control Board (CCB) should be documented.

✦ *Las acciones, roles y responsabilidades del Comité de Revisión Técnica (ERB) y el Comité de Control de Cambios (CCB) deben estar documentadas.*

Cada actor del proceso de cambio debe tener claridad sobre su función y autoridad.

Regla 5

” Cita

Change control must be a primary focus of the program and must be integrated into the culture of the project.

✦ *El control de cambios debe ser una prioridad y estar integrado a la cultura del proyecto.*

El SCM no es un agregado; debe estar inmerso en el día a día del desarrollo.

Regla 6

” Cita

All information that is placed under CM control or promoted in the CM hierarchy must be promoted based on successful completion of a quality gate.

✦ *Toda información que se ponga bajo control de CM o se promueva jerárquicamente debe hacerlo tras pasar por un punto de control de calidad.*

Solo se avanza en el control cuando hay evidencia de calidad. Nada se "sube" si está roto

Regla 7

” Cita

All proposed changes to CM-controlled products should be properly classified.

✦ *Todos los cambios propuestos a productos bajo control deben clasificarse adecuadamente.*

Los cambios se deben **categorizar (bug, mejora, refactor, etc.)**, para tratarlos correctamente.

Regla 8

” Cita

All CM library structures or releases from the CM library must be recorded in a Software Version Description (SVD) document or comparable document.

✦ *Todas las estructuras de biblioteca de CM o entregas deben registrarse en un documento SVD (Descripción de Versión de Software).*

Es necesario tener un documento que describa **qué versión contiene qué, para qué sirve, cuándo se liberó.**

Regla 9

” Cita

It is essential that CM be continually aware of the exact status of products under control, the relationships between these products, problems, and open issues, and who is using the products.

✦ *Es esencial que la gestión de configuración conozca en todo momento el estado exacto de los productos bajo control, sus relaciones, problemas abiertos y quién los está utilizando.*

SCM debe tener visibilidad completa del sistema. **Estado, dependencias, fallas, usuarios. Todo.**

Regla 10

” Cita

The worst way to establish a CM system is to purchase a tool set and then devise a process to fit the tools. Establish the CM process that fits your project and culture, then find a tool set that supports that process.

✦ *La peor forma de establecer un sistema de configuración es comprar herramientas y luego forzar un proceso que las encaje. Primero definí el proceso, y luego elegí las herramientas que lo soporten.*

¡No al “tool-first” approach! Primero se diseña el proceso de SCM, **luego se eligen las herramientas** que lo potencien (no al revés).

Corolario 1

” Cita

Artifacts used for evaluating CM effectiveness must be products of the system. If the CM tool set (manual, automated, or some combination of both) does not produce the records necessary to verify CM integrity, the tool set is flawed.

✦ *Los artefactos utilizados para evaluar la efectividad del CM deben ser productos del sistema. Si el conjunto de herramientas no produce los registros necesarios para verificar la integridad del CM, la herramienta está mal.*

Si el sistema de SCM no puede auditarse, no sirve. La herramienta debe generar trazabilidad y evidencias.

Corolario 2

” Cita

Only the most trivial development can survive without automated tools.

✦ *Solo los desarrollos más triviales pueden sobrevivir sin herramientas automatizadas.*

Un SCM sin autorización no escala. Es esencial automatizar lo repetible y auditable

Corolario 3

” Cita

A mature, stable CM system (process, procedures, and supporting tool set) does not require an army to administer it. A few good staff members should be able to run any development.

✦ *Un sistema de configuración maduro y estable (proceso, procedimientos y herramientas) no requiere un ejército para administrarlo. Un buen equipo reducido puede manejar cualquier desarrollo.*

Si el sistema de SCM está bien pensado, no necesita sobrecarga humana, solo personas formadas.

Filosofía Ágil. Manifiesto Ágil

📄 Información

¿Qué es el agilismo?

El **agilismo** no es una metodología ni un proceso, sino una **filosofía de pensamiento**, un conjunto de **valores y principios** que guían cómo debe ser el desarrollo de software (y otros productos) en **entornos complejos, cambiantes e inciertos**.

- Es una ideología basada en principios, no un marco prescriptivo
- Surge como un equilibrio entre nada de proceso y demasiado proceso
- Es adaptable en lugar de predictivo
- Es centrado en las personas en lugar de los procesos
- Requiere disciplina, aunque sea flexible
- Se basa en retroalimentación, ciclos cortos y mejora continua

Ágil es la **capacidad de adaptarse al cambio, responder a la incertidumbre y entregar valor de forma incremental y sostenida**.

🔧 Tip

Mitos comunes sobre agilismo

- ✗ *“Ágil significa sin planificación”*: FALSO. Ágil tiene planificación, pero continua y adaptable.

- ❌ “*Ser ágil es hacer todo por pedacitos*”: FALSO. No se trata de fraccionar tareas, sino de **entregar valor funcional desde el inicio**.
- ❌ “*En ágil no se documenta*”: FALSO. Se documenta lo necesario y con propósito claro.

¿Qué es un proceso empírico?

Un **proceso empírico** es aquel en el que:

- No se puede predecir el resultado exacto con certeza.
- Se basa en la **observación, la experiencia y la experimentación**.
- Se adapta constantemente en función de la **retroalimentación del entorno**.

Tip

Ágil ≈ Proceso empírico

La filosofía ágil es un claro ejemplo de proceso empírico, ya que:

- Se ajusta constantemente según la **retroalimentación del cliente y del equipo**.
- No sigue un plan rígido, sino que lo **adapta de forma iterativa**.
- Se basa en **entregas funcionales y revisión continua**.

Información

Un **proceso empírico** es aquel que basa sus decisiones en la **observación**, la **experiencia previa** y la **experimentación constante**, en vez de seguir un plan rígido predefinido.

En este contexto, el **agilismo** toma los principios del **proceso empírico** y los **aplica al desarrollo de software y productos**:

- Se basa en ciclos de inspección y adaptación continua (iteraciones).
- El equipo **observa los resultados, reflexiona y ajusta** su forma de trabajar.
- **No se fija en cumplir un plan, sino en adaptarse al cambio** para entregar valor real.

Cita

“At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.”

Manifiesto Ágil

Valoramos más:

- **Individuos e interacciones** sobre procesos y herramientas.
- **Software funcionando** sobre documentación exhaustiva.
- **Colaboración con el cliente** sobre negociación contractual.
- **Responder al cambio** sobre seguir un plan.

No rechazamos lo segundo, damos más prioridad a lo primero.

12 Principios del Manifiesto Ágil

- **Satisfacer al cliente** mediante entregas tempranas y continuas de software con valor.
- **Aceptar cambios** incluso en etapas tardías.
- **Entregar software funcional frecuentemente**, en semanas, no meses.
- **Colaboración diaria entre negocio y desarrollo.**
- **Construir proyectos en torno a personas motivadas.**
- **Conversación cara a cara** como el método más eficiente de comunicación.
- **Software funcionando es la principal medida de progreso.**
- **Desarrollo sostenible**, a ritmo constante e indefinido.
- **Atención continua a la excelencia técnica y buen diseño.**
- **Simplicidad**: maximizar la cantidad de trabajo no hecho.
- **Equipos autoorganizados** generan mejores resultados.
- **Reflexión periódica** del equipo para mejorar procesos y relaciones. Ajustan su comportamiento

Principios que reflejan el pensamiento empírico

- El **Principio 12** (retrospectiva y mejora continua) es el más claro reflejo de un enfoque **empírico adaptativo**.
- El **Principio 2** (aceptar cambios) y el **7** (software funcionando como medida) refuerzan la necesidad de **evaluar continuamente** lo que se construye.

Ágil es una filosofía, no una metodología

Información

Ágil **no es un proceso, ni una metodología**. Es una **forma de pensar**, una **ideología**, un conjunto de **valores y principios** que guían cómo se desarrolla

software (y otros productos).

Valores de los equipos ágiles

Algunos valores prácticos en equipos ágiles:

- **Planificación continua** (a distintos niveles)
- **Equipos auto-organizados y empoderados**
- **Entregas iterativas y frecuentes**
- **Prácticas técnicas disciplinadas**
- **Integración continua y pruebas frecuentes**
- **Priorización basada en valor**

Requerimientos Ágiles

Definición

Los **requerimientos ágiles** son necesidades o funcionalidades que deben cumplirse para entregar valor al negocio y al usuario final, y se gestionan de manera incremental, colaborativa y priorizada.

En Agile (y, en particular, en Scrum) no se trabaja con un documento de requisitos cerrado al inicio, sino con un **backlog de producto**: una **lista emergente y ordenada de lo que se necesita para mejorar el producto**, única fuente de trabajo del equipo, que se **refina de manera continua** agregando descripción, orden y tamaño a medida que aprendemos más del producto y sus usuarios . El Product Owner **maximiza el valor** y gestiona ese backlog haciéndolo **transparente, visible y comprendido** . Scrum, como marco, **envuelve** prácticas (como las historias de usuario) sin prescribirlas en detalle, para resolver **problemas complejos mediante soluciones adaptables** .

User Stories vs Requerimientos ágiles

No son exactamente lo mismo, pero **sí** son la técnica preferida (y más difundida) para **expresar requerimientos de usuario de forma ligera, comprensible y planificable**.

Mike Cohn define las historias como **piezas relativamente pequeñas de funcionalidad valiosa para usuarios** (típicamente 0,5 a 5 días de desarrollo) .

Además, Cohn dedica un capítulo a **manejar requisitos no funcionales** con historias (p. ej., performance, seguridad), lo que deja claro que **las US pueden cubrir más que “features”** cuando se formulan bien .

¿Cómo se expresan los requerimientos? → User stories

Una historia de usuario es una breve descripción de una funcionalidad desde la perspectiva del usuario final. Tiene tres componentes: quién lo necesita, qué necesita y para qué lo necesita.

- Usar el valor para construir el producto correcto
- Usar historias y modeos para mostrar que construir
- Determinar que es "solo lo importante"

3C

Cohn recoge la famosa aliteración de Ron Jeffries: **Card, Conversation, Confirmation**.

- **Card (tarjeta)**: el soporte visible, pero **no lo más importante**.
- **Conversation (conversación)**: donde se **descubren y afinan** los detalles entre cliente/PO y equipo.
- **Confirmation (confirmación)**: **tests de aceptación** que confirman que la historia fue implementada correctamente .
Rachel Davies lo resume muy bien: **las tarjetas representan los requerimientos del cliente más que documentarlos**; los detalles se trabajan en la **conversación** y se registran en la **confirmación** .

Además, Cohn subraya que **las historias no son obligaciones contractuales**; lo clave es conversar en el momento oportuno y capturar expectativas como **criterios de aceptación**

¿Por qué usar historias?

El propio libro de Cohn lista estas razones:

- **Comunicación verbal y comprensibilidad**.
- **Tamaño adecuado** para planificar.
- Funcionan bien con **desarrollo iterativo**.
- **Aplazan el detalle** hasta que es necesario
- **Apoyan el desarrollo oportunista** (aprovechar oportunidades).
- **Fomentan el diseño participativo**.
- **Acumulan conocimiento tácito** en el equipo.

Cómo escribir buenas historias (regla INVEST + formato)

Cohn populariza INVEST a través de sus cualidades: **Independent, Negotiable, Valuable, Estimatable, Small, Testable**

Formato recomendado:

Como [rol], **quiero** [función] **para** [beneficio].

Y **criterios de aceptación** claros (la **C** de Confirmation), que luego facilitan la prueba de aceptación.

El costo del enfoque tradicional BRUF (Big Requirements Up Front)

Es la práctica de intentar **especificar exhaustivamente los requisitos al inicio**. La crítica ágil es que **sobre-planificar temprano** crea desperdicio, porque ni los estimados ni las prioridades son estables tan lejos del inicio.

Más esfuerzo inicial no te da precisión perfecta y, de hecho, **forzar “requisitos firmados” y mucho análisis upfront** suele llevar a planes rígidos... y aun así con estimaciones imperfectas .

- **80% de las funcionalidades** agregadas a productos son **rara vez o nunca usadas**

BRUF cuesta porque:

- **Congela decisiones temprano** cuando la incertidumbre es alta.
- **Induce a firmar largas especificaciones** y a estimar en falso, con retornos decrecientes del esfuerzo de análisis upfront .
- **Empuja a construir funcionalidades de baja/ninguna utilización**, generando inventario de software que no agrega valor (64% rara vez o nunca usado) .

Gestión Ágil de requerimientos de software

Backlog y prioridad por valor

En Scrum, los “requerimientos” viven en la **Pila de Producto (Product Backlog)**: una **lista emergente y ordenada de lo que se necesita para mejorar el producto**; es la **única fuente de trabajo** del equipo .

El **Product Owner (PO)** es responsable de **maximizar el valor** y de la **gestión eficaz del backlog**: crear y comunicar ítems, **ordenarlos** y asegurar transparencia/visibilidad/comprensión .

- **Los requerimientos cambian** y eso es una **ventaja** si puedes actuar sobre ellos: el marco Scrum es empírico y **permite ajustar el Product Backlog en cualquier momento** en función del aprendizaje y del entorno (revisión e inspección frecuente) .
- **Cada iteración implementa lo de mayor prioridad**: el trabajo de cada Sprint se **selecciona desde lo más prioritario** del Product Backlog (tema “Qué” del Sprint Planning) .
- **La prioridad la establece el PO** (con input del equipo), y **el backlog puede agregarse o re-priorizarse en cualquier momento** .

- **Granularidad:** los ítems “de arriba” están más listos/detallados; el **refinamiento** es continuo para **agregar descripción, orden y tamaño** . Además, los ítems “listos” (suficientemente entendidos) **están preparados para ser traídos a un Sprint** tras el refinamiento .

Historias de usuario como técnica central de gestión

- **“Historias ≠ ERS”:** Cohn dedica un capítulo a lo que las historias **no son** (no son IEEE-830, no son casos de uso) y **por qué usarlas:** favorecen **comunicación verbal, comprensibilidad, tamaño adecuado para planificar, iteración, aplazar detalle, desarrollo oportunista y conocimiento tácito**; además, **“Why Not Stories?”** discute cuándo **no alcanzan solas** (p. ej., contextos de fuerte cumplimiento/trazabilidad) .
- En Scrum, usar historias **encaja naturalmente:** el trabajo del Sprint se **elige del backlog priorizado**, se **demuestra con software funcionando**, y el backlog **crece y cambia** con el aprendizaje .

Conversación, una pieza clave

En el modelo **3C** (Card, Conversation, Confirmation), Mike Cohn deja claro que la **tarjeta (Card)** es lo más visible pero **no lo más importante**; los **detalles** de la historia se descubren en la **Conversation** y se **verifican** con la **Confirmation** (tests de aceptación) . De hecho, “las palabras escritas son imprecisas” y **documentar no garantiza** que el cliente obtenga lo que quería; la conversación es lo que ajusta expectativas y significado compartido .

Scrum apoya esto desde su base **empírica** (transparencia, inspección y adaptación) y al promover **interacciones frecuentes:** Daily Scrum, debates durante el día, Sprint Review como sesión de trabajo, etc. . Incluso el propio Cohn te invita a **hablar con usuarios** y usar la conversación para descubrir necesidades reales—no “capturarlas” como si ya existieran fijas .

Tipos de requerimientos

- **Requerimiento de negocio** (objetivo/valor): expresa la **necesidad y metas del negocio** en el **dominio del problema** (no técnicos). SWEBOK explica que los requisitos **expresan necesidades y restricciones** que satisfacen un problema real, típicamente **desde el dominio** y en términos de negocio/operación, no de tecnología .
- **Requerimiento de usuario:** “qué quiere lograr el usuario” en lenguaje del **dominio** (libro, autores, transacciones... no protocolos). Se mantienen **alto nivel** para validación con stakeholders antes de bajar a detalle técnico .
- **Requerimientos de software / del sistema:** traducen lo anterior a especificaciones para el sistema (ya en el **dominio de la solución**). Sommerville distingue

claramente **requisitos del usuario** vs **requisitos del sistema** y muestra ejemplos de cada uno .

- **Funcionales**: servicios/comportamientos que el sistema debe proveer (qué hace ante entradas, situaciones) .
- **No funcionales (calidad/constraints)**: restricciones de rendimiento, seguridad, fiabilidad, estándares, etc.; suelen aplicar **a todo el sistema** .
- **Implementación / proceso** (constraints de desarrollo): p. ej., “debe correr en X” o “se usará Ada”; SWEBOK los llama **parámetros de proceso** o **requisitos de proceso** .

User Stories Applied, Cap. 6–7 (reformular hacia valor; técnicas para conversar con usuarios) .

Triangulo Negocio-Usuario-Software



- **Arriba (Negocio, Usuario) = Dominio del Problema**: necesidades, objetivos, lenguaje del negocio para validar alcance y trade-offs con stakeholders .
- **Abajo (Software/Sistema) = Dominio de la Solución**: especificaciones que el sistema debe cumplir, descompuestas y priorizadas (pueden incluir modelos y criterios cuantificados) .

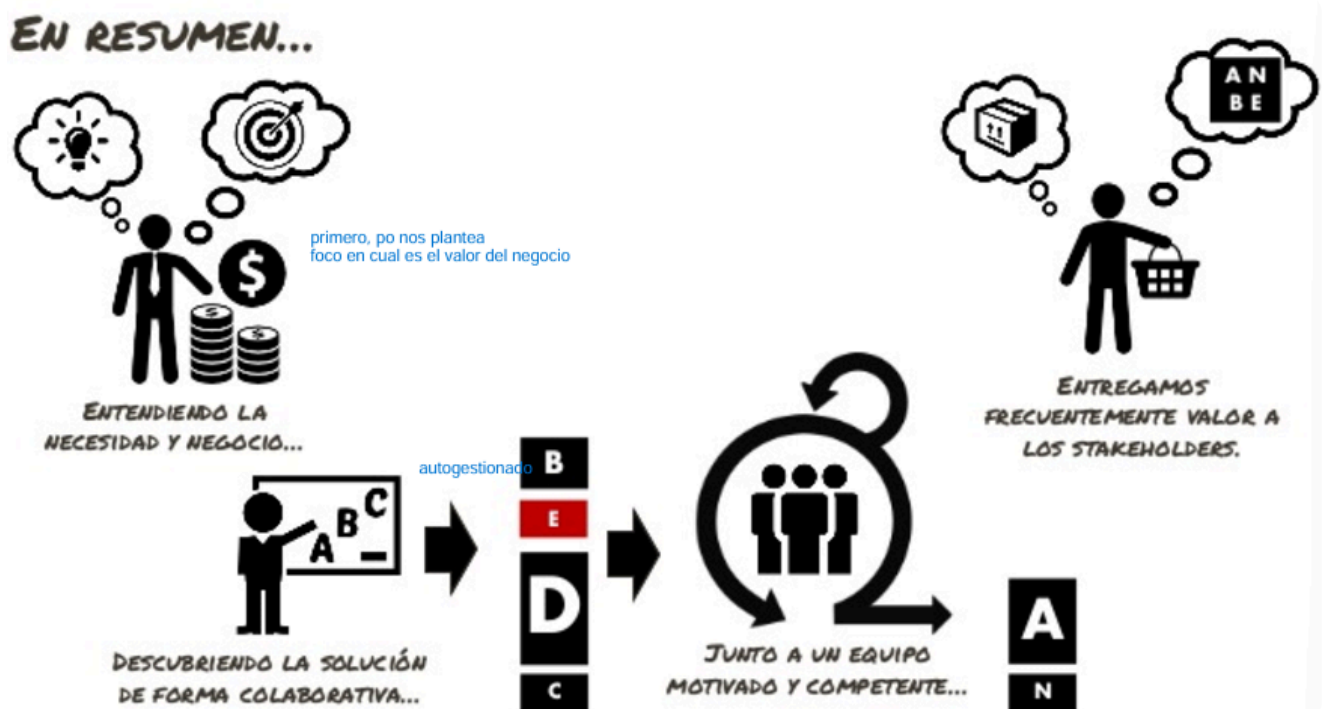
Este pasaje del problema a la solución es precisamente lo que la Ingeniería de Requisitos sistematiza, con **modelos conceptuales, negociación y priorización** antes de invertir fuerte en construcción .

Triangulo de hierro, **Tradicional (plan-driven) vs Ágil (value-driven)**



- **Dirigido por plan:** intenta fijar **requisitos completos al inicio**, derivando plan y asignación de recursos. Cohn critica que una ERS estilo IEEE-830 empuja a una **lista de “shall (deberes)”** que rara vez captura las metas reales del usuario y **lleva a soluciones prematuras**; “por mucho análisis upfront (de antemano), los usuarios cambian de opinión cuando ven el software” .
 - **Dirigido por valor (Ágil): tiempo y equipo fijos, alcance emergente** (se decide “qué entra” por valor en iteraciones). Cohn lo explica con la analogía de la **carrera cronometrada**: sabemos **cuándo** terminamos, no exactamente **qué** entregaremos; planificamos y **revisamos objetivos** a distintos horizontes (release, iteración, día) .
- Y Scrum lo soporta con su **ciclo empírico** (inspección/adaptación) y el **Product Backlog emergente y ordenado por valor** .

Imagen de resumen



- Entendiendo la necesidad y el negocio
Acá aparece el Product Owner (PO) clarificando **qué problema vale la pena resolver y por qué**. Se habla en términos de **objetivos de negocio** (reducción de

costos/tiempos, ingresos, cumplimiento). Clave: foco en **valor**, no en funcionalidades aún.

- Descubriendo la solución de forma colaborativa
Con el PO, usuarios y el equipo se **conversa** (3C: Card–Conversation–Confirmation), se exploran opciones (A, B, C...) y se formulan **historias de usuario** con criterios de aceptación. El énfasis es en **diálogo** y aprendizaje, no en documentos eternos.
- Backlog priorizado por valor
Las letras (A, B, C, D, E) representan ítems del **Product Backlog**. Se **ordenan** por valor, riesgo y aprendizaje esperado. Lo de arriba se entiende mejor (más granular) y está “listo” para entrar al sprint. Ese bloque rojo puede sugerir el **siguiente ítem prioritario**.
- Equipo motivado, competente y autogestionado
En cada sprint el equipo **toma del tope** del backlog, planifica, construye y valida un **incremento** potencialmente liberable. El ciclo circular indica **iteración + empirismo** (transparencia, inspección, adaptación).
- Entregamos frecuentemente valor a los stakeholders
Al final de cada sprint se **muestra software funcionando**, se recoge **feedback** y, con lo aprendido, el PO **reordena** el backlog. Así se maximiza valor y se evita construir cosas de poco uso.

Principios ágiles relacionados a los requerimientos ágiles



1. Prioridad: satisfacer al cliente con entregas tempranas y frecuentes

La idea central es planificar de forma **iterativa y continua** para ir buscando valor (no un plan fijo). Cohn explica que el objetivo del planeamiento ágil es “una búsqueda de valor” que se responde **iterativamente e incrementalmente** (qué construir, con qué recursos y en qué fechas) y resume que el plan ágil **favorece el cambio** y se **distribuye a lo largo del proyecto**.

2. Aceptar cambios de requerimientos incluso tarde

Cohn lo liga directamente con la naturaleza incierta de los proyectos y el **cono de incertidumbre**: los planes deben **poder cambiar** y refinarse con el avance. También

recuerda por qué falla la planificación tradicional: muchos features incluidos en productos **se usan rara vez o nunca (64%)**; por eso se prioriza y replantea continuamente.

4. Negocio y técnicos trabajando juntos todo el proyecto

Cohn abre su libro de historias diciendo que **los requerimientos son un problema de comunicación** entre quienes quieren el software y quienes lo construyen, y que el proyecto fracasa si domina un solo lado; necesitamos una forma de **trabajar juntos**.

6. El medio de comunicación más efectivo es cara a cara

En “Ten Principles for Agile Testers” se desprende del Manifiesto que un principio clave es **“Enable face-to-face communication”**. Esto conecta directo con tus diapositivas del “canal de comunicación”: cuanto más rico el canal, mejor la efectividad.

11. Las mejores arquitecturas/diseños/requerimientos emergen de equipos autoorganizados

El mismo capítulo enumera **“Self-organize”** entre los principios prácticos para equipos ágiles, reforzando que la calidad de decisiones (incluidos requerimientos) **emerge del equipo** que colabora de cerca y se autoorganiza. *Esto en The Scrum Team en User Stories Applied For Agile Software*

User Stories: qué son, 3C y por qué importan

El libro *User Stories Applied* describe que las historias ofrecen un proceso de requerimientos “de punta a punta” para **ahorrar tiempo, eliminar retrabajo y conducir a mejor software**. En el **Cap. 13 “Why User Stories?”**, Cohn resume razones:

- **Enfatizan la comunicación verbal,**
- **Son comprensibles para todos,**
- **Tienen el tamaño justo para planificar,**
- **Apoyan el desarrollo iterativo,**
- **Permiten diferir detalle,**
- **Favorecen diseño oportunista y participativo,**
- **Construyen conocimiento tácito.**

Las 3C: Card, Conversation, Confirmation (lo más importante es la conversación)

Cohn cita a **Ron Jeffries**: una historia se compone de **Tarjeta, Conversación y Confirmación**.

- La **tarjeta** es sólo el recordatorio visible; **“representa” requerimientos, no los documenta** (Rachel Davies).
- La **conversación** es la clave para **definir y refinar** el significado.

- La **confirmación** se plasma en **tests de aceptación** que dicen cuándo la historia está hecha.

Es la conversación, no la nota en la tarjeta, y las historias no son obligaciones contractuales. Además, recomienda dejar el detalle en la parte de atrás como pruebas y mantener el frente de la tarjeta como recordatorio de conversación + dudas.

Información

Las historias son

- Una necesidad del usuario
- Una descripción del producto
- Un ítem de planificación
- Token para una conversación
- Mecanismo para diferir una conversación

El product Owner prioriza el product backlog

En Scrum, el Product Backlog es **la única fuente de trabajo** para el producto y **está ordenado**; el **Product Owner (PO)** es responsable de su **ordenación** (prioridad) y de **maximizar el valor** que entrega el equipo. Además, el **refinamiento** agrega “descripción, orden y tamaño” a los ítems de backlog de manera continua (por eso los de arriba suelen tener **más detalle** que los de abajo)

Los ítems pueden **agregarse o re-ordenarse en cualquier momento** y el backlog se **evoluciona** constantemente. En Scrum, esto se materializa en que, antes de cada Sprint, se **seleccionan las historias de mayor prioridad** que entren en la capacidad del equipo.

“User stories: porciones verticales” (vertical slices)

Cohn recomienda **partir historias grandes en rebanadas “de punta a punta”** del sistema en lugar de dividir por capas técnicas. A esto lo llama **“slice the cake”**: cada historia “debe tener un poquito de cada capa” (UI, lógica de negocio, persistencia) para que **cada historia entregue valor utilizable por sí sola**.

Las historias “rebanada vertical” **reducen riesgos** al ejercitar todas las capas y permiten **liberar** valor aun con funcionalidad parcial.

Consejo

No es “hacer primero la base de datos y después la pantalla”, sino **entregar pequeñas capacidades completas**.

Modelado de roles, Personas y “Personajes extremos”

Cohn dedica un capítulo a **User Role Modeling**: identificar distintos **roles de usuario**, describir sus **atributos** y (cuando ayuda) crear **personas** con nombre, foto y rasgos que vuelvan **concretas** las diferencias entre usuarios. Las **personas** son “representaciones imaginarias de un rol de usuario” usadas para pensar y escribir historias desde su perspectiva.

Cohn también sugiere (como técnica opcional) considerar “**personajes extremos**” (ej.: *el Papa, una mujer con varios novios, un traficante*) para descubrir historias que de otro modo no veríamos; sirven para **generar ideas**, aunque no siempre esas historias terminarán en el producto

“Usuarios representantes / Proxies”: por qué **no son ideales**

Cohn es claro: **lo vital es incluir usuarios reales** en el equipo del cliente; cuando no se puede, se recurre a **proxies** (gerentes de usuarios, marketing, analistas, soporte, etc.) pero hay que ser **cauto** porque **sus prioridades y usos no son los de un usuario típico**.

Conclusión: **sí, “usuarios representantes” = user proxies** (cap. 5). Úsalos solo cuando sea necesario y **valida** con usuarios reales en cuanto puedas.

Criterios de aceptación

- Son condiciones claras que **delimitan el alcance** de una historia y **expresan la intención** (qué debe cumplirse para decir “listo”) sin imponer la solución técnica.
- Cohn los trata como “**expectativas**” del cliente que conviene **anotar en la tarjeta (al dorso)** para que los desarrolladores sepan **cuándo han terminado** la historia: “las expectativas se capturan como pruebas/criterios en la parte de atrás... el objetivo es transmitir información adicional para que los desarrolladores sepan cuándo están hechos” .
- Esas expectativas/criterios ayudan a evitar “ir demasiado lejos” o “quedarse corto” en una historia .
- Definen una intención, no una solución
- Ej.: El usuario debe elegir al menos una cuenta para operar
- Son independientes de la implementación
- Relativamente de alto nivel, no es necesario que se escriba cada detall

Pruebas de aceptación

- Son los **ejemplos concretos, verificables (preferentemente automatizables)** que materializan esos criterios.
- Cohn lo resume así: las pruebas de aceptación **expresan detalles resultantes de la conversación, documentan supuestos, y proporcionan criterios para decidir si**

una historia está completamente implementada; además, deben **escribirse antes** de programar y, en lo posible, **por el cliente** (con ayuda del equipo) .

Relación y diferencias

- **Origen:** ambos salen de la **conversación** PO–equipo; los criterios son el **qué**; las pruebas son el **cómo verificar** ese qué. Cohn insiste en capturar esos detalles como pruebas/criterios cuando se discute la historia, y también al inicio de la iteración antes de programar .
- **Nivel de detalle:** criterios = **alto nivel / intención** (“la búsqueda no debe demorar más de 30s”); pruebas = **casos concretos** (inputs/outputs, casos que pasan/fallan) que “dicen si está completo” .
- **Forma:** criterios pueden ir como bullets en la tarjeta; pruebas pueden ser **tablas ejecutables** (FIT/FitNesse) o escenarios equivalentes (ATDD/BDD) .
- **Responsabilidad:** el **cliente/PO** define las pruebas de aceptación con apoyo del equipo; cualquiera puede ejecutarlas, idealmente automatizadas .
- **Momento:** mejor **antes de codificar**; guían el desarrollo y evitan omisiones (ej., contemplar una compra rechazada)

Algo más sobre las User Stories...

- No son especificaciones detalladas de requerimientos (como los casos de uso)
- Son expresiones de intención, “es necesario que haga algo como esto...”
- No están detallados al principio del proyecto, elaborados evitando especificaciones anticipadas, demoras en el desarrollo, inventario de requerimientos y una definición limitada de la solución.
- Necesita poco o nulo mantenimiento y puede descartarse después de la implementación.
- Junto con el código, sirven de entrada a la documentación que se desarrolla incrementalmente después

¿Story, Épica y Theme: en qué se diferencian?

- **User Story:** unidad pequeña de valor para el usuario/cliente. Sirve para planificar y completar dentro de una iteración. Cohn subraya que las historias son del tamaño adecuado para planear y que podemos refinar un conjunto a través de iteraciones.
- **Épica (Epic):** una historia **grande**. Se usa cuando aún no conviene entrar al detalle; más adelante se “rompe” en historias pequeñas.
- **Theme:** conjunto de **historias relacionadas** que podemos tratar como una sola unidad para estimar o planificar un release.
Cohn también advierte que las épicas y los themes reducen esfuerzo de estimación pero conllevan **más incertidumbre** que estimar historias chicas.

Recomendación práctica: lo cercano se detalla en historias pequeñas; lo lejano puede quedarse como épica/theme.

Spikes: qué son, para qué sirven y cuándo usarlos

Un **ítem de investigación** (investigative story) para **reducir incertidumbre o riesgo** (técnico, funcional, de usabilidad, etc.). No busca entregar una funcionalidad final al usuario, sino **aprender** para poder decidir o estimar mejor. Cohn sugiere incluso **colocarlos en una iteración distinta** de las historias que dependen de ese aprendizaje, para no mezclar una iteración con demasiada incertidumbre.

Cuándo usarlos:

- Cuando hay dudas serias sobre **tecnología, integración, rendimiento o viabilidad**.
- Cuando no está claro **cómo debería interactuar** el usuario con el sistema y necesitamos un **prototipo** o experimentación previa.
- Cuando una historia es **riesgosa** y antes de comprometer una estimación necesitamos evidencia (esto encaja con la priorización por valor/riesgo de Cohn).

Entregables típicos: resultados del spike (un doc corto con hallazgos/decisiones, un prototipo desechable, cifras de performance, alternativas comparadas). La “definición de listo” para la historia afectada debería incluir “spike completado y conclusiones incorporadas”.

Tipos (práctica común):

- **Spike técnico:** investiga enfoques/tecnologías (p.ej., “¿Kafka aguanta 10k msg/s con nuestra topología?”).
- **Spike funcional/UX:** explora con usuarios/prototipos la mejor interacción o reglas de negocio (p.ej., “¿Dashboard con histograma u otra visualización para consumo de energía?”).

(La etiqueta de “técnico” vs “funcional” es práctica extendida en XP/ágil; en los textos de Cohn lo verás como “investigative story/spike” con la recomendación de aislarlo en otra iteración)

Límites/lineamientos útiles:

- Tener un **objetivo explícito de aprendizaje** y un **tiempo acotado** (timebox).
- **Criterio de aceptación del spike:** una **pregunta respondida** o una **decisión tomada**, no “software listo”.
- No mezclar en la misma iteración demasiadas historias dependientes del spike (evita volatilidad de alcance)



Consejo

- **Story** entrega valor de usuario y es *commit-able* dentro de un sprint; **spike** entrega **conocimiento**, no valor directo al usuario final.
- **Épica** es “story grande” (se partirá más adelante); **theme** es un **grupo** de historias. Un **spike** no es un nivel de abstracción, es un **tipo de PB item** para **aprender**.
- En la planificación: épicas/themes ayudan a **estimar a alto nivel** con más incertidumbre; spikes ayudan a **reducir** esa incertidumbre antes de comprometer historias grandes o arriesgadas.

Estimación Ágil y Tradicional

Tips clave sobre estimación

- **No confundir estimación con compromiso o meta.** Una estimación es una predicción; un “target” es un objetivo de negocio; un compromiso es una promesa con fecha y alcance. Mezclarlos da lugar a malentendidos y planes poco realistas. **Cohn también señala que en enfoques tradicionales “las estimaciones se convierten en compromisos” y eso falla con frecuencia.**
- **El valor está en el proceso de estimar (colaborativo), no en un número “perfecto”.** Cohn recomienda involucrar a todo el equipo al estimar y planificar; eso mejora la calidad y el compromiso del plan .
- **Expresar incertidumbre.** Ningún plan es 100% cierto; expresa rangos ya sea en fecha o en funcionalidad . McConnell refuerza esto con el **Cono de Incertidumbre**: al inicio la variabilidad puede ser de $\pm 4x$ y se reduce conforme tomamos decisiones y eliminamos variabilidad .
- **Re-estimar y ajustar continuamente.** El cono no se “estrecha solo”; hay que forzarlo removiendo incertidumbre (definir/recortar alcance, interfaz, requisitos) .

Estimación ágil vs tradicional

Tradicional: plan detallado al inicio, intento de fijar alcance, fechas y recursos; cuando cambian las condiciones, el plan se des-sincroniza con la realidad (se ignora la incertidumbre) .

Ágil: planificación iterativa, se acepta la incertidumbre y se reduce mediante entregas frecuentes y feedback; se planifica a distintos niveles (release/iteración/día) y se pone el énfasis en *prioridad de negocio* y *aprendizaje continuo* .

Ágil

- Las features/stories son estimadas usando una medida de tamaño relativo conocida como story points (SP)
- Las medidas relativas no son absolutas.

Story Points no es una medida basada en tiempo

Estimación relativa

Información

- Las personas no saben estimar en términos absolutos
- Somos buenos comparando cosas
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual
- Se emplea mejor el tiempo de análisis de las storys

¿Qué son los Story Points y por qué son “relativos”?

- **SP = medida de tamaño/Esfuerzo relativo, no tiempo.** Se usa para estimar el “tamaño” de historias comparándolas entre sí ($A \approx 2 \times B$). Cohn lo presenta así en el cap. 4 (“Story Points Are Relative”).
- **Separar tamaño y duración.** Estimamos **tamaño** en puntos y derivamos **duración** usando **velocidad** (puntos/iteración).
- **Velocidad corrige errores.** Si la velocidad real difiere de la supuesta, el plan “se autocorrige” sin re-estimar cada historia.

¿Cómo estimar en ágil?

- **Estimación como equipo + Planning Poker.** Cohn describe técnicas y por qué Planning Poker funciona (reduce sesgos, promueve discusión y convergencia).
- **“Triangulación” / comparación.** La base es comparar historias con referencias conocidas (estimación relativa)
- **Niveles de plan:** release (meses), iteración (semanas), diario (día), con retroalimentación constante.

Tamaño en Agile

Es una medida del **tamaño relativo** de una historia (no tiempo). Con story points medís *cuán grande/complicada/riesgosa/incierta* es una story respecto de otras. Cohn lo presenta así: **“Story Points Are Relative”** y la **velocidad** traduce esos puntos a calendario más adelante.

Estimar de forma **relativa** es más fácil y temprana que estimar en **absoluto**; los story points “se calculan antes y más simple” porque comparás historias y, tras unas iteraciones, el equipo mejora su puntería.

Tamaño ≠ esfuerzo (ni tiempo)

Descripción del trabajo no es igual a *esfuerzo*. Diferentes habilidades, experiencia, supuestos y familiaridad hacen que el **tiempo** sea inestable para comparar. Por eso **separamos**: comparamos **tamaño** (SP) y dejamos que la **velocidad** capture el ritmo real del equipo (no “horas ideales” rígidas) .

Cómo armar el tamaño

- **Escala**: camiseta (S-XXL) o **Fibonacci** (0,1,2,3,5,8,13...). Elegí **una** y **no la cambies** (si cambiás la escala, cambiás la unidad patrón).
- **Factores que influyen**: **complejidad**, **esfuerzo percibido** y **duda/riesgo**. Esa “duda” es justo lo que el ágil explicita y reduce con spikes y aprendizaje continuo.

Dónde entra “lo tradicional”

El **enfoque tradicional** suele estimar **en términos absolutos** y de **tiempo/esfuerzo** directamente, apoyándose en **tamaño absoluto** y **modelos paramétricos** , o en **juicio experto/descomposición/analogía**

Velocity

En planificación ágil, primero estimamos **tamaño** (puntos de historia) y de ahí **derivamos duración** usando la **velocidad** del equipo. Cohn lo resume así: si el total del proyecto son 100 puntos y la velocidad histórica del equipo es 10 puntos por iteración, la duración esperada son 10 iteraciones; la velocidad “corrige” errores de estimación porque, al observarse empíricamente en las primeras iteraciones, ajusta el plan sin reestimar historias

La velocidad es la **suma de los puntos** de las **historias completadas** en una iteración. “Completada” en Scrum significa que forma parte de un **Incremento** y **cumple la Definición de Hecho (DoD)**. La Guía Scrum también explica que el equipo mejora la **confianza de su pronóstico** basándose en **rendimiento pasado** (capacidad + DoD).

Poker Panning

Qué es y por qué funciona

Es la técnica que Cohn recomienda para estimar: combina **opinión experta, analogía y desagregación** y se apoya en la discusión grupal, que mejora la calidad de las estimaciones

Quién participa

Participan **todos los desarrolladores** (programadores, testers, UX, DBAs, etc.). El **Product Owner participa**, responde preguntas, **pero no estima**

Pasos resumidos

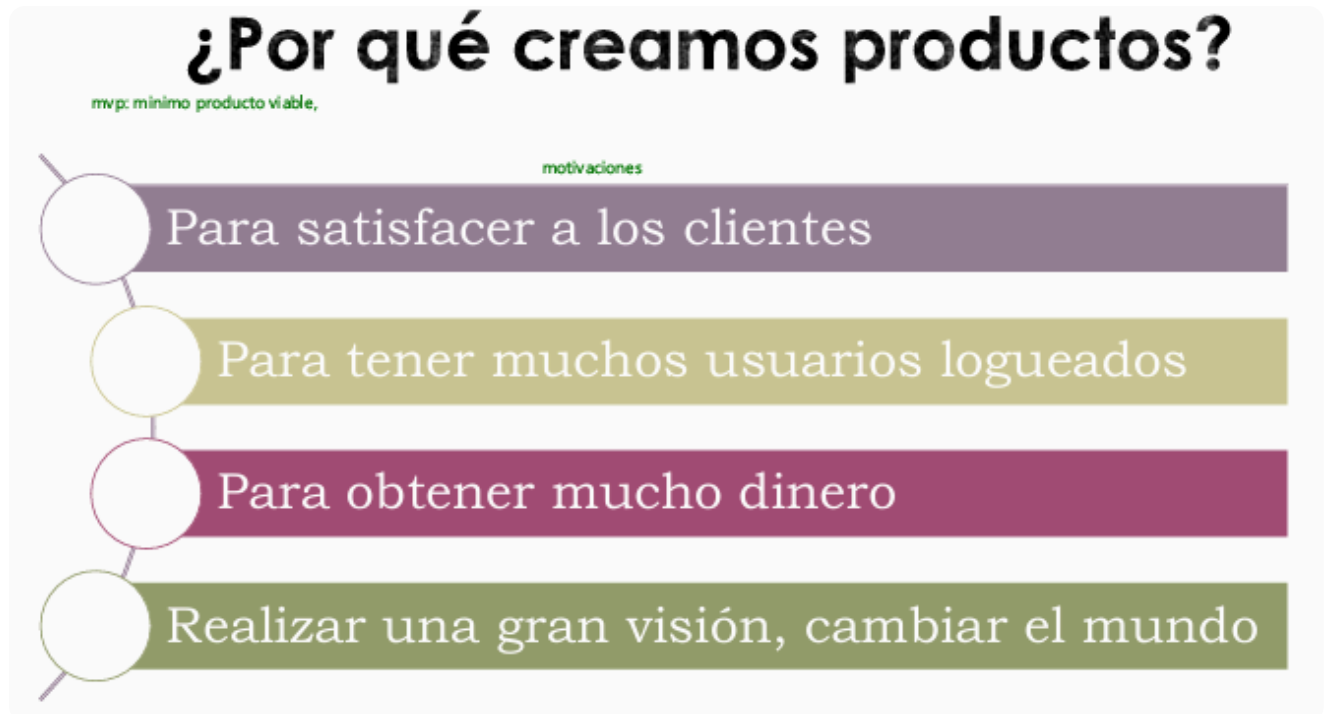
1. El moderador **lee la historia** y el PO aclara dudas.
2. Cada quien elige **en privado** su carta; se **revelan simultáneamente**.

3. Si hay diferencia, hablan **alto y bajo** explicando por qué; se vuelve a votar hasta converger.

Cuándo usarlo

- En **refinamientos** o **planeación** para estimar historias y temas (themes) de forma colaborativa y rápida.

Gestión de Productos



Diccionario rápido

UVP (Unique Value Proposition): la **hipótesis de valor único** de tu producto/servicio (lo que promete ser “distinto”). Es el punto de partida de las láminas: “comprender que un producto nuevo tiene una hipótesis de valor único” (UVP).

MVP (Minimum Viable Product): el **mínimo producto viable** para **probar la hipótesis UVP** con usuarios reales. Incluye lo esencial para que sea **usable/viable** (a veces un poco de “table stakes”) y, sobre todo, sirve para **aprender** si hay “fit” con el mercado inicial (pioneros)

- Si las respuestas de usuarios son **dispersas** (“para usarlo necesitaría X distintos por cliente”), **no hay mercado aún**; toca **revisar** la hipótesis cliente/problema/solución (pivot) y construir un **MVP2** sobre lo aprendido .
- Si el **MVP2** funciona (tracción inicial), pensás cómo crecer y profundizar la adopción más allá de pioneros, en función del **feedback** y la investigación de producto .

MMF (Minimum Marketable Feature): **característica mínima comercializable**; la “pieza mínima” que **aporta valor** y puede **empezar a traer crecimiento** ahora. Se usa

cuando en un área ya hay **fuerte indicio de valor**: sabés qué hacer y podés liberar una **característica pequeña pero lanzable** (optimiza *time-to-market* y permite progresar por pasos) .

- Si tu **primer MMF** “apesta”, volviste a **tierra de hipótesis**: cambiás el foco a una **característica** (no al producto completo) y validás de nuevo porque hay **alto potencial** pero **alta incertidumbre** .

MVF (Minimum Viable Feature): característica mínima viable. Es como un “mini-MVP” **a nivel de feature**: una versión pionera de una funcionalidad para uso real y aprendizaje. Si “pega”, la expandís con **varios MMF**; si no, pivotás a otra característica con mejor potencial

✧ Resumen

UVP (hipótesis) → **MVP1** para validarla → si no valida, **pivot** a **MVP2** → con tracción, creces por **MMF** en áreas de valor → para “abrir camino” en una característica con alta incertidumbre, usás una **MVF** (mini-MVP de feature) que luego se escala con **MMF** sucesivos .

- **MVP**: cuando todavía **no sé** si la **propuesta de valor** encaja. Objetivo: **aprender rápido** con el mercado inicial (pioneros) y decidir si **pivotar** o **perseverar** .
- **MVF**: cuando una **característica** concreta promete mucho pero aún hay **incertidumbre**; busco **evidencia rápida** en esa funcionalidad puntual (mini-MVP de feature) .
- **MMF**: cuando en un área ya hay **alta certeza de valor**; busco **crecimiento** liberando **características pequeñas pero comercializables** (varios MMF encadenados) .

MVP – Minimum Viable Product (Producto Mínimo Viable)

La idea (Lean Startup) es construir la versión más simple de un *producto nuevo* que permita **aprender con clientes reales con el menor esfuerzo posible**. La meta central del MVP es **validar hipótesis** (problema, solución, propuesta de valor) con evidencia y realimentación temprana. En términos de planificación ágil, encaja como un “hito” de un plan de releases: definimos qué historias/temas llegan a ese primer corte y lo vamos ajustando con lo que aprendemos en cada iteración

MMF – Minimum Marketable Feature (Característica Mínima Comercializable)

Es la **pieza de funcionalidad más pequeña** que por sí sola **aporta valor visible** (interno y/o para usuarios) y **podría liberarse**. Piensa en un “trozo vertical” completo (UI + lógica + datos) que resuelve algo valioso. Varias MMF pueden componer un release. (Este concepto se usa mucho cuando priorizamos por valor y riesgo para decidir **qué va primero** en los próximos sprints y releases

MMP – Minimum Marketable Product (Producto Mínimo Comercializable)

Es el **primer lanzamiento de producto** que reúne **las MMF necesarias** para que “tenga sentido” para un segmento de usuarios (early adopters). A diferencia del **MVP** (que busca **aprender**), el **MMP** busca **“ganar” en el mercado**: ya es **vendible/usable** por un grupo objetivo. Operativamente, el MMP suele ser “Release 1” de tu plan.

MMR – Minimum Marketable Release (Release Mínimo Comercializable)

Es un **release** con el **conjunto mínimo de características** que **aporta nuevo valor** respecto de la versión anterior. En un roadmap, verás una secuencia MMR1, MMR2, ... donde cada uno agrega MMF priorizadas. En planificación ágil esto se materializa en el **plan de release**, que se actualiza cada iteración con progreso real (velocidad) y cambios de alcance

Consejo

1. Exploras/validas **hipótesis** con un **MVP** (aprendizaje validado).
2. Priorizas **MMF** (valor, costo, riesgo, aprendizaje) para el **primer release**.
3. Ese **primer release** con suficiente valor es tu **MMP**.
4. A partir de ahí vas haciendo **MMR** sucesivos que agregan nuevas **MMF**.
5. El plan de releases se **revisa y ajusta** continuamente (planificación progresiva).

Errores comunes

- **Confundir MVP con MMP/MMF**
 - **MVP ≠ ganar mercado**: si sólo “aprende” pero **no** está listo para clientes, **no** es MMP.
 - **MVP ≠ maqueta vacía**: debe permitir **aprender algo crítico** con usuarios reales.
- **Enfatizar “mínimo” y olvidar “viable”**
 - Si lo “mínimo” compromete la capacidad de **probar valor** (calidad/uso aceptable), el experimento falla (no aprendes nada confiable).

- **Saltar del MVP al producto completo**
 - La práctica ágil sugiere **iterar releases** (MMR) y **repriorizar** en base a lo que aprendemos y a la **velocidad** del equipo.
- **No priorizar con criterios claros**
 - Usa factores de **valor, costo, aprendizaje y riesgo** para decidir qué MMF entra primero.

Valor vs Desperdicio

- **Valor** = beneficios reales para el cliente; **todo lo demás es desperdicio** (tiempo, coordinación, rehacer, etc.) .
- Una guía rápida: pregúntate “¿*haríamos más de esto si pudiéramos?*”. Si la respuesta es **no**, probablemente es desperdicio .
- Mucho “desperdicio” en software es **carga de falla** (defectos, mala usabilidad, performance pobre) que impide capturar el valor ya “pagado” por el producto; reducirla libera capacidad para nuevas funciones de negocio .

Lean Startup: Build–Measure–Learn

- El circuito **Build → Experiment/Measure → Learn** alinea el trabajo con necesidades reales y te da “dosis de realidad” continua .
- El éxito **no** es “entregar” sino **entregar algo que la gente usa**; por eso el loop sirve para descubrir necesidades y ajustar el rumbo de forma metódica

Construir un MVP

Arranca lo antes posible: el MVP puede ir desde un **smoke test** muy simple hasta un **prototipo temprano**.

Buenas prácticas de “build + smoke test” diarios sustentan esta cadencia: el build debe pasar un **smoke test** end-to-end y mantenerse actualizado a medida que crece el sistema .

Al decidir *cuán complejo* hacer un MVP: **usa juicio, simplifica en la duda**, y **evita construcción/promesas excesivas**; el objetivo es iniciar el ciclo de aprendizaje con el mínimo necesario

Prepara el MVP con pasos concretos: **nicho → roadmap realista → competencia → pre-venta → testear supuestos → asegurar problema correcto → focalizar funciones principales** .

Características a cuidar desde el día 1: diseño aceptable/UX, *usabilidad* suficiente para que la gente quiera usarlo, *confiabilidad* para que early adopters confíen, y *funcionalidad* justa para resolver el problema y evaluar siguientes pasos .

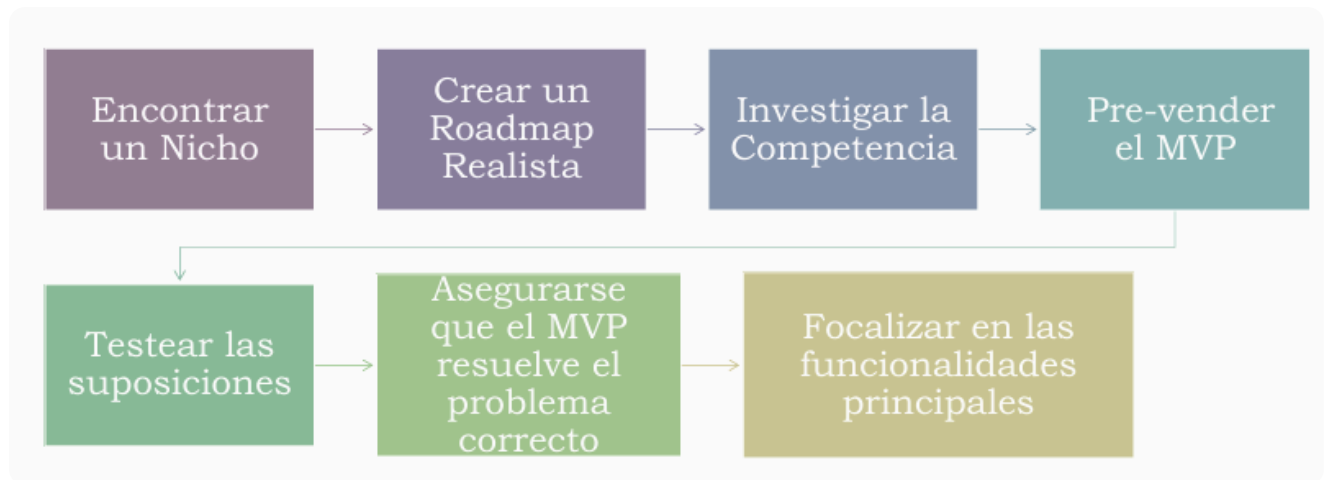
La audiencia de cero

- Muchas veces es más fácil conseguir inversión con **cero ingresos/clientes/tracción** que con “un poco de cada cosa”. Eso incentiva **postergar** lanzamientos hasta “estar seguros”... y es peligroso: sube el **desperdicio**, pierdes **feedback esencial** y aumentas el riesgo de construir algo que **nadie quiere** .
- Antídoto: **usa el MVP para experimentar en pequeño** con primeros usuarios y **valida los supuestos más riesgosos primero** .

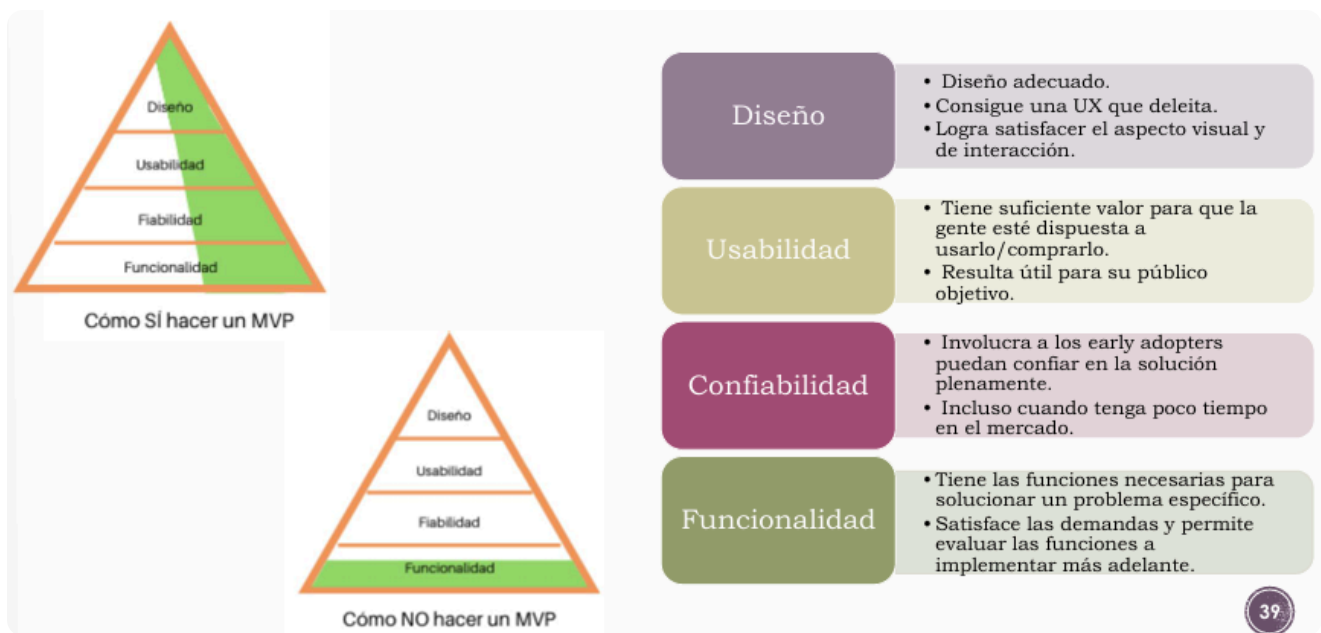
Supuesto de "salto de fe"

- Identifica los **supuestos más riesgosos** de tu idea (los que, si fallan, todo cae). Ej.: en Dropbox, que la **sincronización** era un problema que valía la pena resolver .
- Valida dos hipótesis:
 - **De valor**: ¿entrega valor cuando lo usan? Métrica típica: **retención** .
 - **De crecimiento**: ¿cómo te descubren nuevos clientes? Métrica: **referencias/NPS** .

Preparar un MVP



Características de MVP



Matriz de priorización para el MVP

- Usa una **matriz simple** (Urgente/Importante) para decidir qué **hacer ahora**, qué **planear**, qué **delegar** y qué **eliminar** del roadmap inicial.

	Urgente	No urgente
Importante	1. Hacer Incluir en el MVP	2. Planear Desarrollar para un lanzamiento beta
No Importante	3. Delegar Considerar integraciones de terceras partes.	4. Eliminar Remover del Roadmap del producto

Componentes de proyecto de desarrollo de Software

Proceso de software

Un **conjunto estructurado de actividades, métodos, prácticas y transformaciones** que un equipo utiliza para **desarrollar y mantener** software y sus artefactos. En la práctica se apoya en **personas** (habilidades), **procedimientos/métodos** y **herramientas**

Proceso Definido

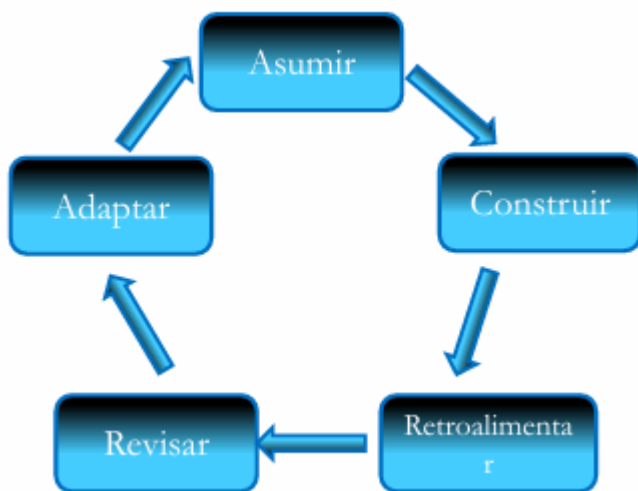
- Supone que el trabajo es **estable y repetible**: si sigo el mismo proceso, obtengo resultados similares.
- La **gestión** se apoya en la **planificación previa** y el **control del cumplimiento** del plan.

- Útil en contextos **poco variables** y con requisitos muy estables.

Proceso Empírico

- Parte de que el trabajo es **complejo y cambiante**; la única forma de controlarlo es **aprender y ajustar continuamente**.
- Se basa en los pilares de **transparencia, inspección y adaptación** (Scrum) .
- La gestión prioriza **ciclos cortos**, evidencia del **valor entregado** y **re-planificación** frecuente (release/iteración/día) .

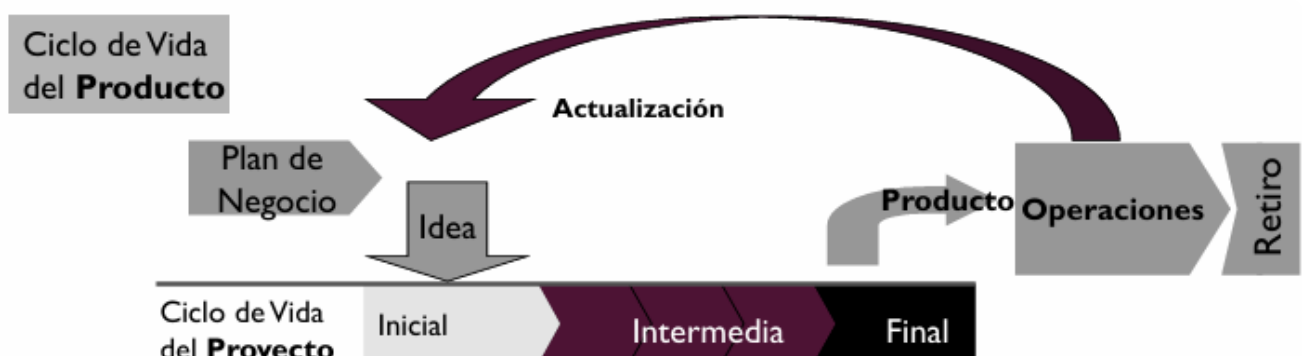
Patrón de conocimiento en procesos empíricos



- **Asumir** (formular hipótesis).
- **Construir** (el mínimo para probar).
- **Retroalimentar/Medir** (recoger datos).
- **Revisar** (aprender).
- **Adaptar** (cambiar rumbo/alcance/prioridad).

Ese patrón es el mismo **Build–Measure–Learn** de Lean Startup aplicado al desarrollo iterativo: **no** buscamos tener razón; buscamos **aprender rápido** y ajustar el plan de producto y de proyecto en consecuencia .

Ciclos de vida: producto vs proyecto



*Ciclo de vida del producto

- Abarca **toda la existencia del producto**: concepción, lanzamientos, operación, evolución, retiro.
- **Más largo**: dentro de él pueden existir **varios proyectos** (replataformado, un gran módulo nuevo, etc.).
- Requiere **roadmap** y **planificación por releases** (MMR, MMP), revisados iteración a iteración con datos reales y velocidad del equipo .

Ciclo de vida del proyecto

- Tiene **inicio → intermedio → cierre**; produce **un incremento** (o conjunto de incrementos) dentro del ciclo de vida del producto.
- En enfoques ágiles, el proyecto se gestiona con **releases e iteraciones**, ajustando alcance/fechas con base en **velocidad** y **prioridad** de negocio .

Tipos básicos de ciclo de vida del proyecto

Secuencial (predictivo “en cascada”)

- Fases en serie (requisitos → diseño → construcción → pruebas → despliegue).
- Encaja naturalmente con **proceso definido**. También se puede ejecutar con prácticas empíricas (p. ej., validaciones intermedias), pero su diseño **favorece** lo predictivo.

Iterativo/Incremental (ágil)

- El trabajo se **parte** en iteraciones; se **inspecciona** y **adapta** en cada ciclo.
- Encaja de forma natural con **proceso empírico** (Scrum/Kanban). Planificación a distintos niveles (producto/release/iteración) y ajuste continuo .

Recurso

- Similar al iterativo, pero con **devoluciones** a etapas previas por diseño (prototipado, exploración técnica). En ingeniería de software moderna suele tratarse como **iterativo/empírico** con **spikes** y prototipos.

Qué es un proyecto

- **Idea base**: es un esfuerzo temporal para lograr un resultado/entrega con objetivos claros, tiempo y presupuesto. En software, la **gestión** (planificar, monitorear y controlar personas, procesos y actividades) es clave para llevar el producto desde una idea hasta la operación completa .
- **Proyecto vs. producto**: el **producto** tiene una vida más larga; varios proyectos pueden alimentarlo con versiones/entregas a lo largo del tiempo (enfoque incremental) .

Características de los proyectos

- **Orientación a objetivos**: guían el trabajo y deben ser claros y alcanzables. En la práctica, el plan documenta **qué/cuándo/cómo/quién** para lograrlos .

- **Duración limitada:** un proyecto termina al cumplir objetivos (temporalidad).
- **Tareas interrelacionadas:** los proyectos combinan esfuerzos y recursos para problemas complejos (complejidad sistémica) .
- **Son únicos:** aun en dominios similares, cada proyecto tiene rasgos propios (contexto, riesgos, personas) — idea consistente con la literatura de gestión de proyectos de software .

Qué es la administración de proyectos

- **Definición práctica:** “tener el trabajo hecho” cumpliendo tiempo, alcance (especificaciones) y presupuesto .
- **Definición académica** (Pressman): La administración del proyecto involucra planificación, monitoreo y control del personal, procesos y acciones que ocurren conforme el software evoluciona desde un concepto preliminar hasta su despliegue operativo completo. .
- **Incluye:** identificar requerimientos, fijar objetivos claros/alcanzables y adaptar especificaciones y planes a interesados (stakeholders) .

La triple restricción (alcance–tiempo–costo) y la calidad

- Objetivos (alcance), tiempo y costos; su **balance** impacta la **calidad** del proyecto, y el líder debe equilibrarlos .
- En enfoque ágil, la calidad **no es negociable:** se recomienda ajustar **alcance** antes que degradar calidad interna (mantenibilidad) para “cumplir plazos” .
- **Práctica ágil típica:** si no alcanza el tiempo, se reduce **alcance** (no se estira el calendario) .

Procesos y ciclos de vida (cómo se relacionan)

- **Proceso de software** (Sommerville): secuencia de actividades (especificación, desarrollo, validación, evolución) que pueden organizarse de diversas formas según el tipo de sistema .
- **Modelos/ciclos** (Pressman):
 - **Secuenciales** (cascada, V): lineales; aplican cuando los requisitos son estables .
 - **Iterativos/incrementales y evolutivos** (prototipos, espiral, concurrente, UP): aceptan el cambio y entregan versiones funcionales rápido; ideales para contextos creativos y requisitos emergentes .

Desarrollo incremental y *release planning*

“Cada nueva versión es desarrollada **incrementalmente** en una serie de pasos”

- En planificación ágil, el alcance por iteración se ajusta con el cliente para cumplir cronograma (pivoteando alcance antes que extender fecha) .

Rol del líder de proyecto

- **Actor/esfera** : interactúa con cliente, subcontratistas, reguladores, gerencias y equipo técnico (coordinación y comunicación transversal) .
- **Equipo de proyecto**: personas comprometidas con objetivos comunes, con habilidades diversas, tamaño usualmente pequeño y fuerte sentido de responsabilidad conjunta (sinergia) .
- Pressman enfatiza la importancia de **equipo**, **líderes** y **coordinación** como conceptos clave del capítulo 24 .

Plan de proyecto (no confundir con roadmap)

- **Qué documenta**: *qué/cuándo/cómo/quién* (alcance, proceso/ciclo, estimación, riesgos, recursos, cronograma, controles y métricas) .
- **Métricas de apoyo** (Pressman): proceso/proyecto/producto para saber “cuándo reír y cuándo llorar” y mejorar de forma racional

Qué es un plan de proyecto

Documento vivo que explica **qué** haremos, **cuándo**, **cómo**, **con qué** y **quién** lo hará, para entregar un resultado con el **alcance**, **calendario** y **costo** acordados.

Objetivo. Alinear expectativas y orquestar trabajo: reduce incertidumbre, coordina personas/recursos, y provee una base para el seguimiento y control.

Cuándo se usa. Se arranca en la **iniciación** del proyecto y se va refinando; en enfoques ágiles es un plan **ligero y adaptable** que se actualiza cada iteración.

Definición del objetivo del proyecto

Usá un objetivo **SMART**

- **Claro**: “TODOS entendemos lo que hay que hacer”. Evitá ambigüedad.
- **Alcanzable**: “Posible con estos recursos y en este tiempo”.

Ejemplo (banco): “En 3 meses, publicar ‘Consulta de saldo’ en web móvil para 50 000 clientes con tiempo de respuesta < 2 s y 99,5% de disponibilidad.”

Definición del alcance

Diferenciá dos planos:

a) Alcance del *producto*

Lista de **características**/restricciones que el producto podrá incluir (lo que en ágil vive en el **Product Backlog** o una SRS en tradicional).

Se mide contra: la **Especificación de Requerimientos** (SRS) o criterios de aceptación de historias (DoD).

b) Alcance del *proyecto*

Es **todo y sólo el trabajo** necesario para entregar ese producto (análisis, diseño, desarrollo, pruebas, despliegue, capacitación...).

Se mide contra: el **Plan de Proyecto** (WBS, hitos, entregables).

Proceso y ciclo de vida

Elegí el **modelo de desarrollo** que mejor calce con riesgo/certeza:

- **Secuencial** (tipo cascada): fases lineales. Útil cuando requisitos son estables y el costo de cambios es alto.
- **Iterativo/Incremental** (ágil): ciclos cortos con feedback, valor temprano y gestión del cambio.
- **Recursivo/Evolutivo**: prototipos, exploración y refinamiento.

Estimación de software

tamaño → esfuerzo → calendario → costo → recursos

- **Tamaño** (de trabajo): p. ej., casos de uso, **Puntos de Función** (tradicional) o **Story Points** (ágil).
- **Esfuerzo**: horas-persona requeridas dado el tamaño, complejidad, riesgo y productividad esperada.
- **Calendario**: esfuerzo distribuido en el tiempo según capacidad/equipo/dependencias.
- **Costo**: calendario × tarifas + licencias/infra.
- **Recursos críticos**: habilidades/cuellos de botella que condicionan el plan (ej.: único especialista en seguridad).

Tradicional vs Ágil

- **Tradicional**: el **PM no “impone”** números; **consolida** estimaciones de *quienes harán el trabajo* (bottom-up con WBS), usa históricos/modelos (COCOMO, puntos de función) y ajusta por riesgos.
- **Ágil**: **estimación relativa y colaborativa** (póker de planificación, triangulación, *reference stories*). Unidad: **Story Points** (tamaño ≠ tiempo). La **velocidad** del equipo convierte puntos → tiempo.

Gestión de riesgos

Un **riesgo** es un “problema esperando ocurrir”. Proceso continuo:

1. **Identificar** (técnicos, de planificación, personas, externos).

2. **Analizar** (Probabilidad × Impacto, matriz, Top-N).
3. **Planear respuesta**: **evitar**, **mitigar**, **transferir**, **aceptar** (con disparadores).
4. **Monitorear y controlar**: revisá el **Registro de Riesgos** en cada hito/iteración; aprendé (*lessons learned*).

Asignación de recursos y organización

- Definí **roles** y responsabilidades (matriz **RACI**).
- Asegurá **habilidades críticas** (seguridad, DevOps, UX).
- En ágil: **equipo multifuncional y autoorganizado**; en tradicional: estructura matricial o por funciones.

Programación (cronograma)

- Estructurá el trabajo con una **WBS**.
- Modelá dependencias (**red PERT/CPM**), camino crítico, holguras, **hitos**.
- En ágil: **sprints**, **releases** y previsión por **velocidad**.
- Siempre incluí **buffers** (incertidumbre ≠ 0).

Controles del proyecto

- **Gestión de cambios** (control de alcance, baselines de *scope/schedule/cost*).
- **Gestión de configuración** (versionado de artefactos).
- **Revisiones** y **gates** (o *reviews* de sprint).
- **KPIs**: EV/EVM (SV, CV), *burn-down/burn-up*, defectos, *lead time*, *cycle time*.

Métricas

- **Proceso**: lead time, *cycle time*, *WIP*, tasa de *rework*, cumplimiento de DoD.
- **Proyecto**: esfuerzo consumido vs plan, **SV/CV**, hitos, **riesgos abiertos**.
- **Producto**: tamaño (PF, historias), **densidad de defectos**, cobertura, rendimiento, NPS.

Monitoreo y Control

- **Definición**. Conjunto de prácticas para **comparar lo planificado vs. lo real**, detectar desvíos temprano y **actuar** (corregir curso, re-planificar o renegociar alcance/costo/fecha).
- **Objetivo**. Mantener el proyecto **predecible** y el producto **útil y de calidad**, protegiendo valor y presupuesto.

“¿Cómo se atrasa un proyecto?” — “De a un día por vez”

- El atraso casi nunca aparece de golpe: surge por **pequeños deslizamientos diarios** no gestionados (tareas que “casi están”, bloqueos sin visibilidad, retrabajo

no registrado, deuda técnica).

- Antídotos: **timeboxing** real, definición de “hecho” estricta, tableros visibles, dailies breves que saquen bloqueos y **terminar** antes que empezar más cosas (limitar WIP)

Tres factores TOP de éxito

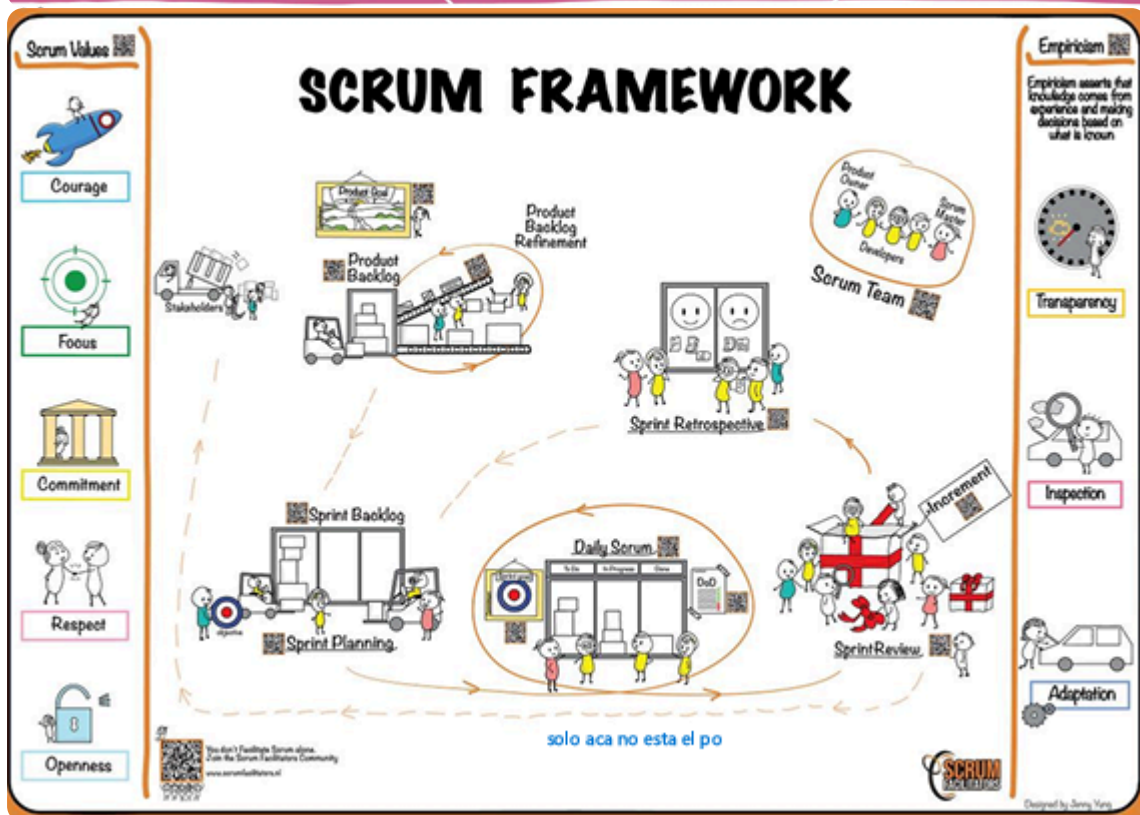
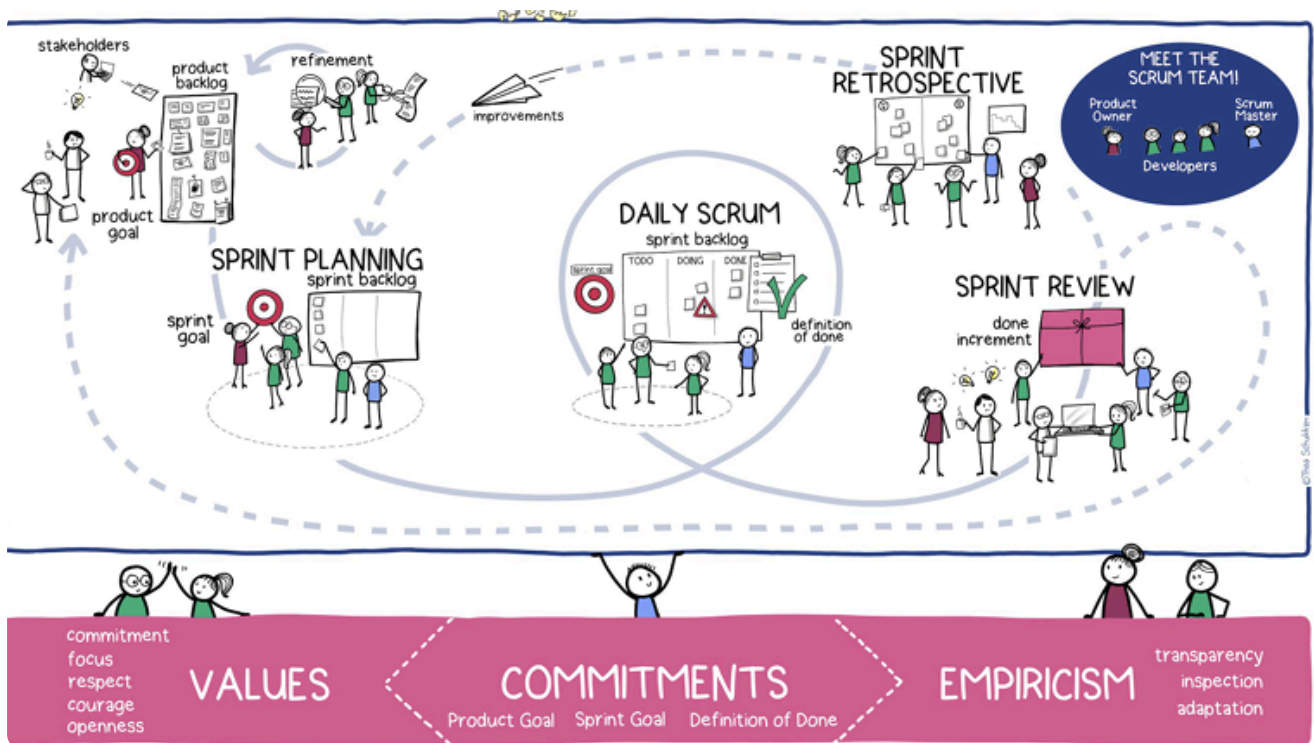
1. **Monitoreo & feedback continuo.** Sin feedback, los desvíos crecen silenciosamente. Con ciclos cortos (scrums/demos/retro) **aprendes y corriges** antes.
2. **Misión/objetivo claro.** Define **prioridad** y trade-offs cuando aparecen conflictos (triángulo de hierro). Evita trabajo inútil y alinea decisiones diarias.
3. **Comunicación.** Reduce “pérdida por transmisión” y bloqueos. Cara-a-cara o video > chat > correo para ambigüedades (regla de la **riqueza del canal**).

Causas frecuentes de fracaso

- **Problema mal definido.** Si el objetivo es borroso, optimizas hacia un destino equivocado (retrabajo masivo).
- **Datos insuficientes para planificar.** Sin históricos ni hipótesis explícitas, subestimas (McConnell: *cone of uncertainty*).
- **Plan hecho “desde planificación” sin el equipo.** Se pierde el **conocimiento local**; baja el compromiso; estimaciones no creíbles.
- **Sin seguimiento del plan.** Un plan sin control es un deseo: no hay detección temprana de desvíos.
- **Plan pobre en detalles.** Sin *work packages* claros ni criterios de “hecho”, aparecen huecos y dependencias ocultas.
- **Asignación de recursos inadecuada.** Cuellos de botella y multitarea destructiva; fechas no alcanzables.
- **Estimaciones basadas en supuestos no validados.** Se ignoran complejidad, riesgos, productividad real.
- **Nadie a cargo.** Falta de **accountability**; los problemas no encuentran dueño ni resolución.

Scrum (ya estamos por finalizar al fin, interminable esto lpm)

Resumen de la imagen



Product Backlog: lo ordena el Product Owner (PO). Es la lista única del trabajo de producto, guiada por el **Objetivo de Producto (Product Goal)**, que es el norte de largo plazo del equipo

Refinamiento: actividad continua donde se agrega detalle, orden y tamaño a los ítems. **Quien talla el tamaño son los Developers**; el PO puede influir ayudando a entender alternativas

Sprint Planning: el equipo acuerda un **Sprint Goal (por qué)**, selecciona ítems (**qué**) y arma un plan para entregarlos (**cómo**). Duración máx. 8 h si el Sprint es de 1 mes. El Sprint Backlog queda como plan vivo "por y para los Developers"

Daily Scrum: evento diario de 15 min para **Developers**; PO y SM solo participan si están trabajando como Developers en ítems del Sprint (el product owner puede no estar) Sirve para inspeccionar el avance al Sprint Goal y ajustar el plan del día.

Incrementos: cada día el equipo avanza hacia uno o varios incrementos **utilizables y aditivos**; un trabajo solo cuenta como incremento si cumple la **Definition of Done (DoD)**

Sprint Review: inspección del resultado del Sprint con stakeholders y decisión colaborativa de qué hacer después; se ajusta el Product Backlog. Máx. 4 h en un Sprint de 1 mes

Sprint Retrospective: el equipo planifica **cómo aumentar calidad y efectividad** (personas, interacciones, procesos, herramientas y DoD)

El siguiente Sprint empieza inmediatamente (la cadencia es continua, 1 mes o menos por Sprint)

Información

Review es “más producto” (decidir próximos pasos y valor con stakeholders) y **Retrospective es “más equipo/proceso”** (mejoras internas y de calidad)

Roles/ Responsabilidades

Scrum Team: un solo equipo pequeño, auto-gestionado, responsable “de punta a punta” de crear un incremento valioso **cada Sprint**

Developers: crean el incremento y son dueños del plan diario y del Sprint Backlog

Product Owner: maximiza valor y gestiona eficazmente el Product Backlog (definir y comunicar Product Goal, ordenar ítems, asegurar transparencia)

Scrum Master: facilita el entorno Scrum y la mejora continua

Artefactos y sus compromisos

Product Backlog → **Product Goal** (objetivo de largo plazo)

Sprint Backlog → **Sprint Goal** (único objetivo del Sprint; da foco y flexibilidad sobre el “cómo”)

Increment → **Definition of Done** (transparencia de calidad; sin DoD no es incremento ni se muestra en Review)

Timeboxes de Scrum

Sprint: 1 mes o menos; es el contenedor de todos los eventos

Sprint Planning: hasta 8 h en Sprint de 1 mes

Daily Scrum: 15 min

Sprint Review: hasta 4 h

Sprint Retrospective: hasta 3 h (en la sección de eventos de la Guía; misma pauta de “proporcionalmente menos” para sprints más cortos)

Pilares y valores que enmarcan todo

Pilares empíricos: Transparencia, Inspección, Adaptación

Valores Scrum: Compromiso, Coraje, Enfoque, Apertura, Respeto

- Estos valores sustentan la colaboración y el cumplimiento de los compromisos de cada artefacto.

DoR vs DoD (la duda frecuente)

- **DoD (Definition of Done):** es **oficial** en Scrum y es el compromiso del Increment. Es un **compromiso del artefacto Incremento**. Define los **criterios de calidad** que cada incremento debe cumplir para considerarse “Hecho”: integrado, probado, documentado según lo que el equipo acordó, y **potencialmente entregable**. La DoD aporta **transparencia** a la calidad y permite medir el progreso real. Ejemplos típicos (como en tu checklist): código en repo, revisado, pruebas (unidad/integración/sistema), documentación de usuario actualizada, defectos conocidos = 0, etc. Si algo de eso falta, **no está Done**.
- **DoR (Definition of Ready):** **no forma parte de la Guía Scrum 2020**. Algunos equipos la usan como práctica complementaria para acordar qué significa que un ítem esté “listo” para ser tomado, pero no es un requisito del marco (útil a veces, solo si no crea burocracia). Es una **lista de verificación previa** para considerar un **PBI** (Product Backlog Item) “listo para planificar/construir” en el sprint. Suele incluir: valor de negocio claro, criterio de aceptación testable, dependencias identificadas, tamaño razonable (estimable y factible en un sprint), riesgos entendidos, etc. La DoR ayuda a reducir ambigüedades en la **Sprint Planning**, pero recuerda: **no es requisito oficial de Scrum**.

Capacidad del equipo en un Sprint

La **capacidad** es el **esfuerzo disponible** para trabajar en PBIs durante el sprint.

Componentes

- **Otras actividades de sprint:** planning, review, retrospective, refinamiento.
- **Compromisos externos:** guardias, soporte, otros proyectos.
- **Tiempo personal/ausencias:** vacaciones, feriados, licencias.
- **Capacidad de trabajo en PBIs:** lo que queda para construir valor.
- **Buffer** (opcional): margen para imprevistos (defectos, bloqueos).

Ambiente de trabajo

- **Abierto y colaborativo:** pizarras/boards visibles, información radiada. Silencio absoluto suele indicar poca interacción.
- **Horarios definidos por el equipo:** facilita ownership y coordinación.
- **Acceso a stakeholders:** PO disponible, usuarios cerca (virtual o presencial).
- **Herramientas livianas:** que no eclipsen la conversación (board físico o digital simple).

Herramientas

Board típico (flujo Kanban dentro del sprint):

- **Story** (fila o swimlane por PBI).
- **Columnas:** *To Do* → *In Progress* → *To Verify/QA* → *Done*.
(Puedes añadir *Blocked* o *Review* si aporta claridad).
- **Tarjetas:** nombre, ID, criterio de aceptación breve, estimación, responsable inicial.

Buena granularidad

- Cada **story** se trocea en **tareas pequeñas** (≤ 1 día ideal por tarea).
- Varias tarjetas avanzan a la vez, el flujo se mueve a diario.

Mala granularidad:

- Una sola tarjeta gigante por story que “no se mueve” varios días. Produce cuellos de botella y falsa sensación de avance.

Sprint Burndown y gráficos de seguimiento

Sprint Burndown (no es obligatorio en Scrum Guide, pero muy usado):

- Eje X: días del sprint. Eje Y: trabajo restante (horas o puntos).
- **Línea ideal** vs **línea real**.
 - Por encima de la ideal = *atraso*.
 - Caídas bruscas = *tareas cerradas al final* (riesgo de “mini-waterfall”).
 - Línea plana = *bloqueos* o tareas demasiado grandes.

Gráficos de producto / release:

- **Release Burnup:** X = sprints; Y = trabajo **hecho** y **alcance total**. Muestra progreso y cambios de scope (línea de alcance se mueve).
- **Product Burndown:** X = sprints; Y = trabajo **restante**. Si el alcance cambia, puede confundir; por eso muchos prefieren *burnup*.
- **Velocidad:** puntos entregados por sprint (ayuda a prever fechas con incertidumbre explícita).

Múltiples niveles de planificación

- **Estrategia/Portafolio**: visión, apuestas, presupuestos, outcomes.
- **Producto**: metas (Product Goal), roadmap de hipótesis/valor.
- **Release**: agrupación de sprints para un hito de negocio (opcional, fuera de la guía pero común).
- **Iteración (Sprint)**: objetivo del sprint, PBIs seleccionados.
- **Día**: *Daily Scrum* para plan diario y gestionar riesgos micro.

¿Qué y cuándo estimar?

- **Portfolio backlog (meses)**: estimaciones gruesas para grandes iniciativas/“épicas”. Útiles para decidir inversión y prioridad. Usa talles (S-M-L-XL) o una secuencia amplia de puntos. Exactitud baja; sirve para comparar.
- **Product backlog (semanas)**: estimaciones relativas por **Item de Product Backlog (PBI)** (p. ej., story points). Permiten ordenar y prever releases. Deben revisarse en el **Refinamiento** (actividad continua).
- **Sprint backlog tasks (días/horas)**: en **Sprint Planning** los Developers pueden descomponer PBIs en tareas y, si quieren, estimarlas en horas. Foco táctico de corto plazo.

Información

Regla práctica: a mayor horizonte, menor precisión. En el Sprint la estimación deja de ser pronóstico y pasa a ser **compromiso con el Objetivo del Sprint** (Guía Scrum 2020: foco en objetivo y plan).

Planificación de Portafolio

- **Envisioning → filtro económico**: canaliza ideas (iniciativas) y, con criterios de valor/costo/riesgo, decide **go/no-go**.
- **Salida**: un **Portfolio backlog** priorizado que alimenta productos y/o líneas de producto.

Planificación de Producto

- **Entradas**: visión/hipótesis, horizonte, presupuesto, apetito de riesgo.
- **Participan**: Product Owner (responsable), Developers, stakeholders clave.
- **Salidas**:
 1. **Product Goal** (compromiso del Product Backlog en Scrum 2020)
 2. **Product roadmap** (opcional, no Scrum “puro”, pero habitual).
 3. **Product Backlog** inicial y criterios para medir valor.

Planificación de Release

- Une **alcance** (rango de PBIs que tiene sentido entregar), **fecha** y **presupuesto** en un plan de hitos (uno o varios Sprints).
- Se revisa con **velocidad histórica** para prever cuántos PBIs entran por release (siempre variable: se optimiza para alcanzar objetivos, no para “llenar la caja”).

Niveles de granularidad

- **Meses:** Épicas / Iniciativas (mayores que un release).
- **Semanas:** Features o PBIs grandes (mayores que un Sprint).
- **Días:** PBIs “listos” para un Sprint
- **Horas:** **Tareas** técnicas para ejecutar PBIs en el Sprint.
- Pasa de grueso → fino a medida que se acerca el trabajo (refinamiento continuo).

Releases y cadencia de Sprints

- **Release tras múltiples Sprints:** patrón clásico de productos con despliegues agrupados.
- **Release tras cada Sprint:** DevOps/continuous delivery.
- **Release por feature:** cuando una característica completa aporta valor inmediatamente.
- En **todos** los casos, **cada Sprint** produce **un Incremento** “Done” potencialmente utilizable (Guía Scrum 2020).

Sprint Planning (planificación de iteración)

La guía lo divide en tres temas:

1. **¿Por qué?** → **Objetivo del Sprint**.
2. **¿Qué?** → PBIs seleccionados para alcanzar ese objetivo.
3. **¿Cómo?** → plan de trabajo (puede incluir tareas).

Salidas: Objetivo del Sprint (compromiso del Sprint Backlog), Sprint Backlog y plan inicial.

Daily Scrum (planificación diaria)

- 15 min, **solo Developers**. Inspeccionan el progreso hacia el **Objetivo del Sprint** y **adaptan** el plan (no es un reporte al PO ni un control de personas).

Métricas en ambientes ágiles (minimalismo)

- **Propósito:** tomar mejores decisiones, no “medir por medir”.
- **Principios guía (Manifiesto/Guía):**

1. **Entregas tempranas y continuas de software valioso.**
 2. **Software funcionando es la medida principal de progreso.**
- **Minimalismo:** elige pocas métricas que conecten con objetivos: p. ej., **Valor entregado**, **Lead time**, **Throughput**, **Calidad (defectos)** y **Satisfacción**. Evita “vanity metrics”.

RTF – Running Tested Features

- Curva acumulada de **features “Done” y probadas** por iteración.
- Mide **progreso real** (software funcionando) y muestra ritmo de entrega (similar a un **burn-up**, pero conteo por funcionalidades).

Capacidad del equipo (Capacity)

Fórmula base por Sprint:

Capacidad = (WH) Horas de trabajo al día × (DA) Días hábiles de la iteración × Miembros disponibles

Ajusta por ausencias/compromisos.

Ejemplo (como tu tabla, 10 días, 6 h/día):

- Capacidad por persona y Sprint = $6 \times 10 = 60 \text{ h}$.
- Sprints 1-2: 4 personas → **240 h**.
- Sprints 3-4: 8 personas → **480 h** cada uno.
- Sprints 5-6: 7 personas (uno se casa) → **420 h** cada uno.
- Sprint 7: 8 personas → **480 h**.

Total = **2 760 h**.

Con esa capacidad y la **velocidad** puedes prever cuántos PBIs entran.

Velocity (Velocidad)

- **Definición:** suma de **story points** de PBIs **Done** en un Sprint.
- **Reglas sanas:**
 - Solo cuenta **Done** (según **DoD**).
 - Se estabiliza tras 3-5 Sprints; úsala para **pronosticar rangos**, no como objetivo de rendimiento.
 - Es **del equipo**, no individual.
 - La velocidad ayuda a **corregir sesgos** de estimación con datos reales.

¿Dónde está la agilidad? (triángulo)

- **Valores** (respeto, apertura, foco, coraje, compromiso) y **pilares** de Scrum (**Transparencia**, **Inspección**, **Adaptación**) sostienen los **principios**.

- Las **prácticas** (p. ej., TDD, pairing, CI/CD) son medios, no fines. La agilidad “vive” en alinear valores+principios para **entregar valor frecuente** y **aprender**.

:p